明镜 · 技术方案（平台版）

> **这一份给谁看**：给 IT 团队。**目标是：新来的后端只给这份东西、不问人，就能开工。** 同时也给车总看——**每一节末尾都写清"这个决定影响什么商业结果"**，不懂技术也能判断。 2026 年 10 月 · 陕西驭界科技有限公司

零、30 秒版

一句话：

> **明镜是"一套系统、三类账号、四个入口"。 技术上要立住三件事：门分得开（账号与身份）、数据漏不出去（隔离与权限）、每一分钱算得清（计量）。**

五个决定（每个都影响商业结果）：

#	决定	一句话理由	影响什么商业结果
1	**一套系统，不做四套**	四个入口不是四个产品，是同一套代码看不同的东西	维护成本不翻四倍
2	**门靠"身份"配，不靠"代码分支"**	旧版教训：「别在代码里写死角色」	将来加第五道门不用改代码
3	**权限只在"数据出口"判断一次**	150 个接口里漏一个就是漏洞	权限做一整套被一个接口绕过 = 白做
4	**双保险隔离 + 一条 CI 测试证明它真的双**	"看起来有两道"和"犯错也漏不出去"是两个估值	尽调必答
5	**每一次 AI 调用从第一行代码就记账**	事后补不了；内测数据就是单位经济模型的第一版	**800 万撑 6 个月还是 18 个月**

三条不许破的边界（破了就出事）：

```
① 能用规则做的，永远不进 AI 层          → 成本差 10 倍以上，只在账单上看得出来
② 前端永远不直连模型                    → 密钥泄露 + 成本记不了账 + 权限被绕开
③ AI 的权限 = 它服务的那人的权限        → 否则 AI 就是绕开全部权限的后门
```

★ 还有一条，比上面三条更"小"，但它是这套系统里最重要的一行配置

```
④ 所有"能不能看见"的默认值，一律取【看不见】那一侧
      anchor_visible        默认 false   → 新加的字段，主播默认看不到
      field_whitelist       默认全禁      → 没显式放行的字段，不可能被返回
```

平台方查询　　默认脱敏　　→ 想拿到明细，必须显式声明
共享缓存　　默认不缓存　　→ 只有标记为"行业知识类"的才进缓存

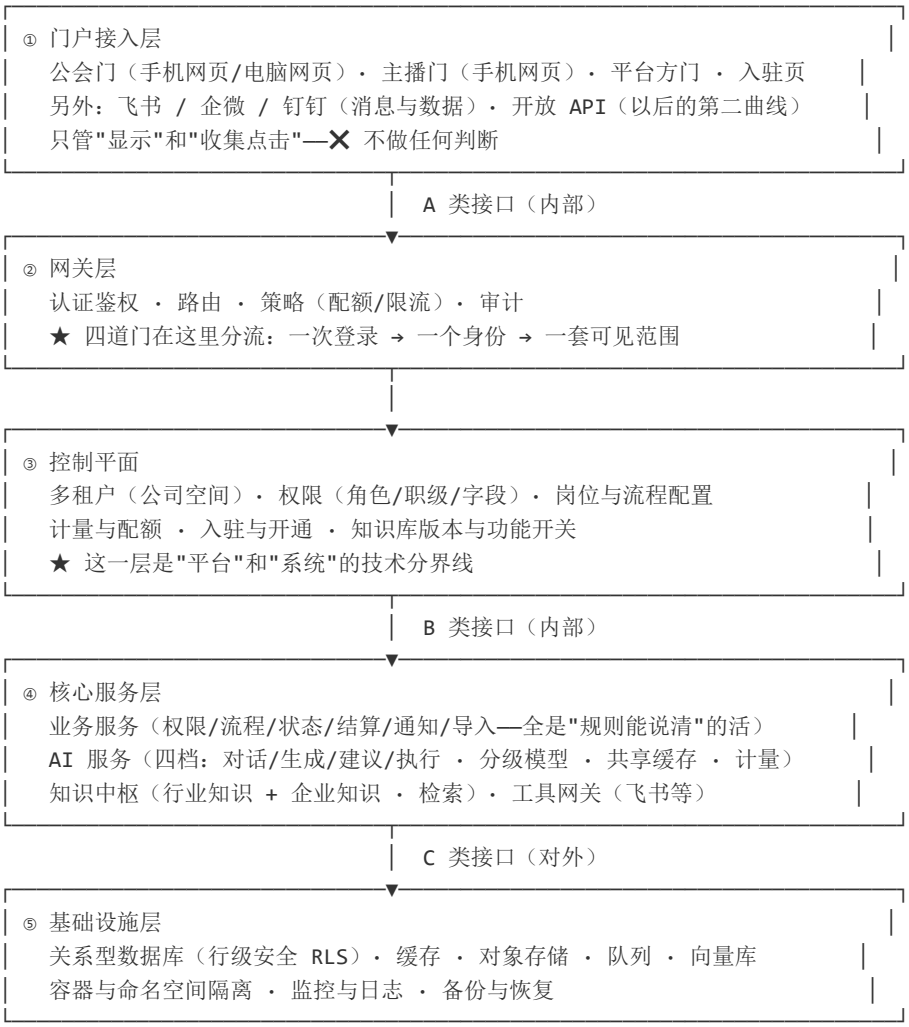
为什么这一条要放在最前面（而不是埋在附录里）：

默认"看得见"	**默认"看不见"（我们）**
有人加了个字段、忘了配 → **不该看的人看到了**	忘了配 → **该看的人少看一个字段**
不可逆（看到了就是看到了，收不回来）	**可逆**（补一次配置就好）

> **一句话：不靠"记得配"，靠"不配就看不见"。 它的价值不在省事，在于把一个"要靠人盯"的系统，变成"犯懒也不出事"的系统。** 具体落地见附录 A.2 ⑦（`field_tag`）和 A.2 ⑧（`field_whitelist`）。

一、总架构（五层）

1.1 一张图



1.2 为什么是五层，不是"随便分层"

每一层的存在都是为了让某件事"只在一个地方做"：

层	它让什么只在一个地方做	如果不分层会怎样
① 门户	显示。**判断不在前端**	前端各自判断权限 → 改一处漏十处
② 网关	**认证 + 分流**。所有入口共用一个登录	每个入口写一套登录 → 会话不通、审计不全
③ 控制平面	**平台级的事**（租户、权限、计量、开通）	平台方需求散落到业务代码 → **加一家客户要发一次版**
④ 核心服务	**"干活"和"花钱"**分开	AI 调用散落各处 → 成本记不了账
⑤ 基础设施	隔离与容量	靠应用层自己保证 → 一次疏忽全线暴露

> ★ **第三层（控制平面）是这一版新增的，也是"平台版"和"系统版"的技术分界线。** 上一版是一套"给一家公司用的后台"——**它没有控制平面，因为只有一家公司。** 有了 1,000 家，**"租户、权限、计量、开通"必须是平台自己的能力，不能写进某个业务模块。**

1.3 影响什么商业结果

做对了	做错了
加一家客户 = 后台点一下（不用发版）	加一家客户要找开发改代码 → **2-3 个商务铺 1,000 家根本不可能**
计量在控制平面统一做	成本散在业务里 → **单位经济模型永远算不出来**
四道门共用一套网关	四套登录 → 审计不全、尽调答不上来

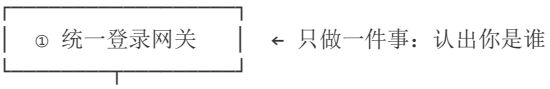
二、四道门在技术上怎么立住（★ 这一节是这一版的重点）

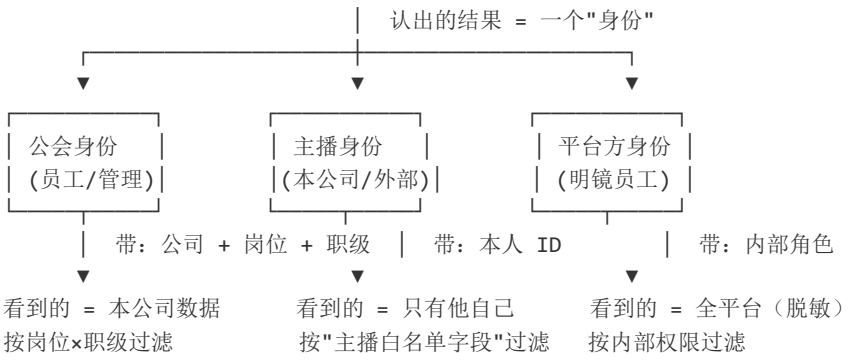
2.1 先破一个最容易做错的思路

❌**错误做法**：四道门 = 四套系统 / 四个前端工程 / 四套账号表。

✅**正确做法**：

> **一套系统，三类账号。门是"身份"决定的，不是"代码分支"决定的。**





2.2 三类账号怎么分（账号表结构）

核心设计："账号"和"身份"分开。

表	装什么	为什么这么分
账号（`user`）	手机号、密码/验证码、状态、创建时间	**一个人只有一个账号**（换身份不换账号）
身份（`identity`）	账号 + 类型 + 归属。三种类型：**公会成员 / 主播 / 平台方成员**	一个账号可以有多个身份（见下面的现实情况）
任职（`employment`）	身份 + 公司 + 岗位 + 职级 + 生效时间	**岗位职级是"数据"不是"代码"**（旧版教训：别在代码里写死角色）

> **表名以附录 A 为准**（`user` / `identity` / `employment`）——本文档只有一处定义，就是附录 A。 这一节讲"为什么这么分"，附录 A 讲"字段长什么样"。

三种身份的差异（技术上的硬差别）：

项	公会成员	主播	平台方成员
绑定公司	✅必须有	✅**可以有，也可以没有**	❌不属于任何客户公司
看数据范围	按岗位 × 职级	**只有他自己名下/他本人的**	全平台，**但脱敏**（只看用量不看内容）
能否看别人的数据	按范围规则	❌**永远不能**	只看汇总，不看明细内容
计费归属	公司	**他本人**（ToC）或公司（ToB 的延伸）	计入平台成本

> ★ **关键的一条现实：一个账号可以有多个身份。** 现实里存在这种情况：**一个人既是某公会的运营，自己也在播**（行业里很常见）。 所以**不能让"账号类型"写死**——账号是账号，身份是身份，一个人可以同时有两个身份，**在登录时选进哪道门。** 这不是为了优雅，是为了**不出现"这个人只能用其中一个身份"的产品事故。**

关于"外部主播"（没有公会的主播）：

他的"归属公司"是空的，**这类账号必须能独立存在**——因为**野生主播是 ToC 的真实客户**（产品方案 7.5）。

2.3 三张表的骨架（字段级定义见附录 A）

```
user 账号
  id                主键
  phone             唯一（登录用）
  status            正常 / 停用 / 注销
  last_login_at     最近登录（用来算"沉默"）
  created_at

identity 身份            ★ 全系统的中枢
  id
  user_id           → user
  kind              member 公会成员 / anchor 主播 / platform 平台方成员
  tenant_id         → tenant   ★ member 必填 / anchor 可空 / platform 必须为空
  status
  ★ 唯一约束：(user_id, kind, tenant_id)   ← 防重复身份

employment 任职          （只有"公会成员"有这一层）
  id
  tenant_id         → tenant   ★ 必须与 identity.tenant_id 一致
  identity_id       → identity
  department_id     → department（可空）
  role_id           → role
  rank              member / lead / director / exec   ← 四条范围规则（见 3.2）
  started_on / ended_on   ★ 离职不删记录，写 ended_on（合规留痕 + 漏斗历史不变）
```

★ **三处"可空/必填"不是随手定的，各有一条理由**：

字段	规则	为什么
`identity.tenant_id` @ member	**必填**	公会成员一定属于某家公司
`identity.tenant_id` @ anchor	**可空**	**外部主播必须能独立注册**——野生主播是 ToC 的真实客户
`identity.tenant_id` @ platform	**必须为空**	**留一个非空的 platform 身份，就是留了一个越权的口子**

2.4 公会门里的"三种视野"——技术上怎么实现

产品上看到的是"同一个门，三种内容"。技术上它是三件事的组合，不是三个界面：

产品上的"视野"	技术上是什么	谁决定
一线员工：**今天我要干哪几件事**	**数据范围 = 本人** + 看板按"该我做的"过滤	任职里的**职级**
组长：**本组今天的活和风险**	**数据范围 = 本组**	职级
管理层：**今天有什么要我拍板的事**	**数据范围 = 全公司** + 多出几个"拍板类"卡片	职级 + 岗位

★ **所以答案是：靠"数据范围"实现，不是靠"视图切换"。**

```
同一个页面模板
        │
        ▼
接口返回数据时 →  范围过滤器按【职级】决定子句：
        本人     →  where owner_id = 我
        本组     →  where group_id in (我的组)
        本部门   →  where dept_id in (我的部门)
        全公司   →  where tenant_id = 我的公司
        │
        ▼
管理层多出来的那几个卡片（掉人归因、签约到期）
        →  由【岗位 + 职级】组合出的"功能可见性"决定
```

为什么不做成三个界面：

三个界面 = **三套要维护的前端 + 三套数据口径。**而"视野"这件事的本质是"看多远"，**它天然是数据层的活。**

> **一句话给 IT：界面只有一套；变的是"接口返回了哪些行、哪些列"。**

2.5 数据边界：主播绝对看不到的那几类，怎么在代码层保证"不可能漏"

产品要求（产品方案 7.1 三条硬纪律）：

#	主播不能看到
1	**评级**（面试评级、培训评级等 5 个角色的评级）
2	**去留状态 / 流失原因 / 淘汰环节**
3	**保底金额**、其他人的任何数据、公司财务与分成

❌**靠"前端不显示"是不行的**——前端拿到的数据在浏览器里，按 F12 就看见了。

✅**四道防线（必须全做）**：

道	做什么	谁保证
第一道：接口分路	**主播端用独立的接口路径前缀**（如 `/me/*`），**不再复用**公会端的业务接口	后端（开发规范）
第二道：字段白名单（默认拒绝）	主播接口的返回字段**用白名单定义**——没写进白名单的字段**不可能被返回。默认全禁，显式放行**	控制平面（字段标签体系）
第三道：数据库行级安全	对主播身份，数据库策略要求 `主体 = 本人`。**就算接口写错了、查询写漏了条件，数据库这一层也返回空**	基础设施（RLS）
第四道：CI 测试（关键）	一条自动测试：**用主播身份去调每一个公会端接口**，断言全部失败；	CI（每次提交自动跑）

	再断言返回的 JSON 里不含禁用字段名	

★ 第四道是这一节的落点：

> **前三道是"设计上不可能漏"，第四道是"证明它真的没漏"。 没有第四道，前三道就只是"看起来安全"。**

要写进字段标签表的"禁用清单"（对主播身份一律拒绝）：

> 评级 / 面试评级 / 培训评级 / 去留状态 / 流失原因 / 淘汰环节
> 保底金额 / 分成比例 / 公司流水 / 他人流水 / 他人联系方式
> 工资 / 绩效 / 组内排名 / 复盘录音原始文件 / 面试评价
>
> **这张清单怎么"落成机制"（不是靠人记）：靠一个默认值。** | 机制 | 在哪 | |---|---| | 任何一个新加的字段，**主播默认看不到** | `field_tag.anchor_visible` 默认 `false`（附录 A.2 ⑦） | | 主播端接口**默认不放行任何字段** | `field_whitelist` 全禁、显式放行（附录 A.2 ⑧） | **换句话说：上面这张清单不需要每次去背——只要默认值是对的，忘了配也不会漏。 反过来，如果默认是"可见"，那么这张清单就要靠每个人每次都记得核对一遍**——那是迟早要出事的做法。

2.6 影响什么商业结果

做对了	**做错了**
主播觉得"这是帮我赚钱的工具" → **愿意用、愿意留、愿意付**	主播发现能看到自己的评级和淘汰环节 → **这是监视我的工具** → 跑掉、还会告诉别人
加第五道门（比如"合作方门"）只需配身份	每加一类用户要改代码、发版
尽调时能说"有 CI 测试证明漏不出去"	只能说"我们很小心" → **两个估值**

2.7 对抗性审查

谁	**会问什么**	**怎么答**
IT 负责人	"三类账号 + 多身份，会不会做复杂了？"	复杂的部分是**一张表的字段**（身份表），换来的是"一个人两个身份"不出事故、加门不发版。**如果写死账号类型，第一个真实案例就会撞上**（既是运营又在播的人）。
IT 负责人	"为什么主播端要独立的接口，不能复用？"	复用省的是开发量，赔的是**"默认能看到"。安全设计的默认值必须是"看不见"**——白名单不是更麻烦，是更安全。
投资人技术尽调	"主播凭什么相信你们？"	**不是靠承诺，是靠机制**：数据库行

		级策略 + 字段白名单 + CI 测试，**三层里任何一层单独成立都不够，三层一起才是"不可能"。**
公会老板	"我公司的人也分三档？"	是，**而且这四档规则（本人/本组/本部门/全公司）是行业里本来就有的层级**，不是我发明的。**客户自己也能改**（岗位和字段标签是数据）。

三、权限怎么落到代码

3.1 一条核心原则（最重要的一条）

> **权限不在业务代码里判断，在"数据出口"统一做。 业务代码只管干活，不管谁能看。**

如果在业务代码里判断	**在数据出口统一做（我们）**
150 个接口，每个都要记得写一遍	**写一次，所有出口都有**
漏一个就是漏洞（漏的概率很高）	漏不了
开发时要分心想"这个接口谁可以调"	开发时只想"把活干完"

> **给 IT 的一句话：业务开发者不需要知道权限怎么算——他只需要知道"我写的查询会自动被收窄"。**

3.2 三层权限怎么落

① 公司层（多租户）
数据访问中间件【自动】注入 "公司 = 当前身份的归属公司"
★ 前端不许传公司 ID，一律从登录态取
★ 开发者不写，条件也会有

② 范围层（看多远）
一张"岗位 × 职级 → 范围"的对照表
范围只有 4 个值：本人 / 本组 / 本部门 / 全公司
例：运营（成员）= 本人 · 运营组长 = 本组 · 运营总监 = 本部门 · 总经办 = 全公司

③ 字段层（看哪几列）
每个字段贴一个标签，三种贴法：
公开 → 全公司都能看（主播昵称、直播间编号、赛道）
范围内 → 按范围规则走（跟播记录、日报、面试记录）
点名 → 只给列出来的岗位（联系方式、保底金额、评级）
数据出口按标签过滤掉不该给这个人看的列

3.3 三条硬要求

#	要求	为什么
1	**岗位、职级、字段标签都是"数据"，不是"代码"**	旧版教训原话：「**别在代码里写死角色**」；客户要能自己配（每家敏感字段不一样）
2	**AI 的权限 = 它服务的那人的权限**	**★ 不能给 AI"全公司"权限——那是绕开所有权限的后门**
3	**每次越权尝试要记日志**	不一定拦住，但**要能查**（合规 + 排查）

第 2 条展开（这是最容易出事的地方）：

```
用户问 AI："公司里谁的工资最高？"
  ├─ ✗ 如果 AI 用"全公司权限" → 它就答了 → 这个人本来看不到
  └─ ✓ 正确做法：把这个人的权限上下文一起传给 AI 层
        → AI 只能查这个人能查的数据 → 它的回答自然就答不出来
```

> **权限体系做了一整套，被一个 AI 接口绕过去——那是白做。**

3.4 影响什么商业结果

做对了	做错了
财务第一次敢把工资数据放进系统（字段级权限）	财务永远在 Excel 里 → **少一个真实使用场景，就少一分数据**
客户敢放敏感数据	客户只放不痛不痒的 → **AI 学不到关键东西**
加岗位、改字段标签不用发版	每个客户都要定制 → **平台退化成外包**

四、多租户隔离（双保险）

4.1 两种做法

做法	像什么	风险
✕靠人记得	所有数据堆一个仓库，每件货贴标签，取货的人要记得只看自己公司的	**一个人忘了，1,000 家客户的数据全暴露**
☑双保险	① 进门时系统自动带你公司的证件 ② **门锁自己也认人**	忘了也漏不出去

4.2 具体怎么落

① 进门验（应用层）
数据访问中间件【自动】给每个查询加上 "公司 = 我的公司"
★ 不是"提醒开发者要写"，是"开发者不写也会有"

② 门锁验（数据库层）
用行级安全（RLS）或等价机制，让数据库自己再检查一遍：
"这条记录的公司，是不是这个连接身份的？"
★ 就算第一道被绕过，第二道也拦得住

4.3 三级组织：平台 → 企业 → 部门

平台（明镜自己）
└ 企业（一家公会 = 一个租户 = 一个数据空间）
└ 部门（运营部 / 经纪人部 / 视觉部……）
└ 人（岗位 + 职级）

隔离等级分两种，客户可以选（这是给大客户准备的）：

方式	怎么做	适合谁
逻辑隔离（默认）	同一个数据库，**行级安全**强制按公司过滤	绝大多数公会（成本低、开通快）
物理隔离（可选）	独立命名空间 + 独立向量库集合	对数据极敏感的大客户

> **一期只做逻辑隔离**，但**数据结构必须一开始就留出"物理隔离"的位置**（每张表都带公司列）。 **因为从逻辑隔离改成物理隔离，是搬家；一开始就想清楚，只是配置。**

4.4 ★ 必须有一条测试来证明它真的"双"

> **设计上双，不等于实际双。**

这条测试做什么：

1. 故意写一个"忘记带公司条件"的查询
2. 断言：它返回的记录数 = 0（而不是返回了别家公司的数据）
3. 再断言：跨公司的聚合查询也被拦住

这条测试必须进 CI（每次提交代码自动跑）。

> **为什么**：**双保险的价值在于"开发者犯错时也不出事"。 如果没有测试证明它拦得住，它就只是"看起来有两道"。**

4.5 还有三条相关的

#	措施	为什么
1	**导出/备份的数据也要隔离**	漏的地方往往不是主库，是"顺手导出的那份"
2	**AI 层拿数据时同样要带公司上下文**	不然 AI 就成了"能跨公司看"的接口

3	**客户可以查"我的数据被谁访问过"**（后期）	尽调友好；也是给客户的安全感

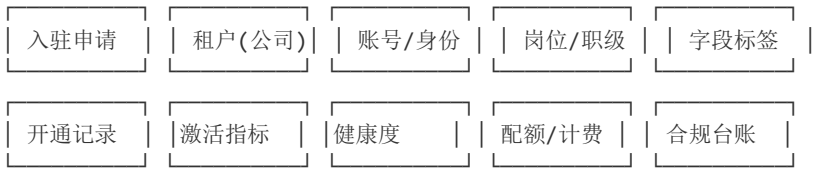
4.6 影响什么商业结果

多租户的安全等级，不是"我们很小心"，是"就算有人犯错也漏不出去"。 一次串号 = 1,000 家客户同时流失 = 前期投入全部打水漂。 而做双保险的成本：很小的开发工作量。这是整份方案里投入产出比最高的一件事。

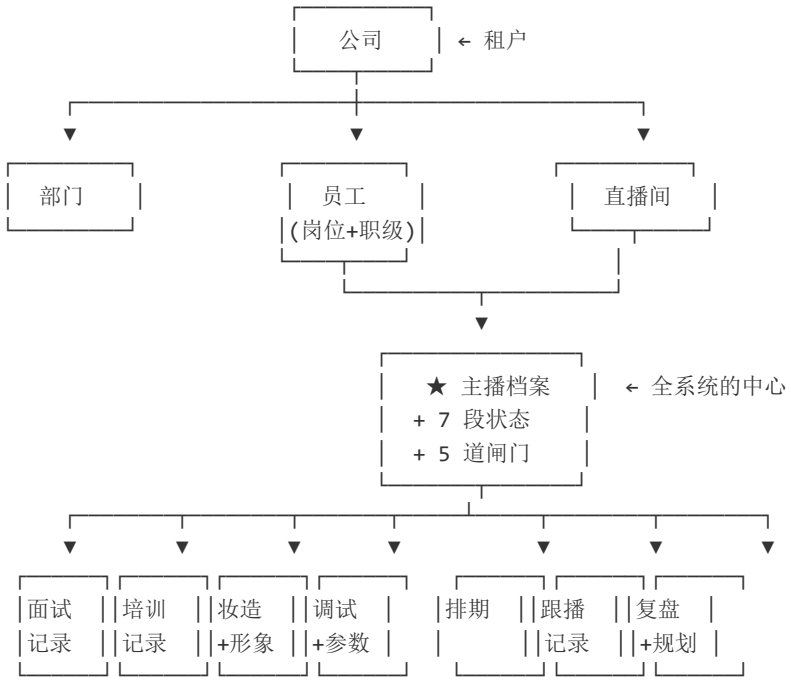
五、数据怎么存

5.1 数据实体总览

① 平台级（控制平面）— ★ 这一版新增，上一版完全没有

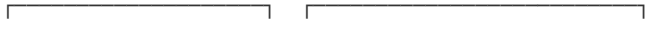


② 企业级（租户内）— 以"主播"为中心（11 个交接点最后都汇到一个人身上）



不挂在主播身上的：拍摄计划 · 设备台账 · 结算记录 · 通知 · 知识库
★ 主播端独立的部分：主播账号资料 · 主播看到的方案 · 主播自己的对话记录

③ 底座（每个系统都要有，明镜必须有）



```
| 操作日志（审计）      |  | ★ AI 调用记录（成本与计量）|
└──────────────────────┘  └──────────────────────────┘
```

5.2 四条数据设计原则

#	原则	为什么
1	**每张业务表都带"公司"列**	双保险的基础；也是将来做物理隔离的前提
2	**"主播"是中心，别处用引用**	11 个交接点都汇到一个人身上——**"一页看完"靠的就是这个结构**
3	**原始数据不覆盖，改动留痕**	数据是资产（BP 的核心故事）；**改了就说不清了**
4	**★ "能用规则算出来的"不存**	存了就会不一致。**能算的算，不能算的才存**

> **第 4 条要展开说**：跟播记录里的"提交率""异常判断"这类，**能由规则算出来的就不要存字段**—— 存了以后规则一改，历史数据就是错的。**存"事实"，不存"结论"**（除了 AI 给出的判断，那个是留痕需要）。

5.3 两个"必须有"的底座

实体	干什么	影响什么商业结果
操作日志	谁改了什么、什么时候改的、原来是什么	**合规留痕**（法规要求的四件事里有"约束行为"和"教育培训"）；**出纠纷时拿得出来**
AI 调用记录	每一次调用：谁、哪家公司、哪一类、多少 token、多少钱	**★ 这是成本控制的唯一抓手**（见第七节）

六、接口怎么分（四类）

6.1 四类接口

类	从 → 到	数量级	特点
A 类	前端 ↔ 后端	最多（**做全了约 150；一期约 64，清单见附录 B**）	**每个都要过网关鉴权 + 数据出口过滤**
B 类	后端 ↔ AI 层	少	**每个都要记账**（三类标记，见 7.3）

C 类	后端 ↔ 客户平台（飞书等）	不多但关键	**决定"能不能铺开"**（冷启动有没有数据）
D 类	数据入口（导入/录入/读客户平台/上传）	少	**"导入"是一等公民**（优先级高于手工填）

> ★ **两个数字的口径**（第 10 轮统一，以后都用这个）： **150 = "做全了的规模"**（每个实体 × 每种操作的估算）；**64 = "一期要做的"**（按一期功能清单收敛，见附录 B.6）。 **凡涉及承诺、排期、人力的地方，一律用"一期 64"；150 只用来表达"这套东西的规模感"。**

6.2 A 类接口的命名与分门（★ 给 IT 的硬规范）

接口路径的第一段，直接对应"哪道门、哪个身份"：

```
/pub/*        公开（登录、入驻申请、健康检查）— 不需要身份
/org/*        公会门（公会成员身份）— 自动带公司上下文
/me/*         主播门（主播身份）— 自动限定"只有他自己"
/platform/*   平台方门（明镜员工身份）— 走内部权限，输出脱敏
```

为什么要这么分（这是 2.5 第一道防线的落地）：

好处	说明
门和接口一一对应，不会串	主播端的代码里**根本不存在**能调到 `/org/*` 的路径
鉴权可以按前缀配	网关一个规则管一类，不用逐接口配
CI 测试好写	"用主播身份调所有 `/org/*`，断言全 401/403"——一条测试覆盖全部门禁

6.3 每个 A 类接口的"标准返回结构"

统一信封（所有接口一致，前端只写一次解析）：

```
{
  "code": 0,                 // 0 = 成功
  "data": { ... },           // 业务数据（已按字段标签过滤过）
  "meta": {
     "scope": "self",        // ★ 这次返回用了哪个范围（调试与审计用）
     "filtered": ["phone"],  // ★ 这次被过滤掉了哪些字段（解释"为什么我没看到"）
     "cost": { "tokens": 0 } // 只有 AI 类接口有
  }
}
```

> ★ **`meta.filtered` 这个设计值得说**： 当运营发现"我看不到保底金额"时，产品上可以直接告诉他"**这个字段对运营岗不开放**"—— **而不是让他以为系统坏了。** 一个字段的"看不到"能被解释，比"看不到"本身重要。

七、钱怎么算（★ "算力中转站"的技术位置）

> **车总的原话**：**「未来他们要付费让 AI 去解决事情，我们属于是算力的中转站。」** 这句话不写进融资材料，**但技术上必须有位置——现在不做，将来补不上。**

7.1 要做到的三件事

① 计量必须准　　每一次调用：花在谁身上、哪个功能上、多少钱
② 计费必须能改　　免费 / 试用 / 收费 / 配额 / 分档 —— 都是配置，不是代码
③ 成本必须能省　　分级模型 + 共享缓存 + 能用规则就不用 AI

7.2 AI 调用记录表（每次调用一条，不可改）

```
AI 调用记录 ai_usage
  id
  发生时间
  公司 id              → tenant        ★ 1 号标记：哪家公司
  调用人 id            → identity      ★ 2 号标记：谁调的
  功能标识             如 "跟播复盘生成" ★ 3 号标记：哪一类（不是"哪个接口"）
  场景类型             对话 / 生成 / 建议 / 执行（四档）
  模型标识             如 "doubao-seed-2.0-lite"
  输入 token 数
  输出 token 数
  计价单价快照         输入单价 / 输出单价      ← ★ 存快照，不存"金额"
  本单成本（元）
  是否命中共享缓存     是 / 否
  耗时（毫秒）
  投递结果             成功 / 失败 / 超时
```

★ 三个字段要特别说明：

字段	为什么这么设计
三个标记（公司 / 人 / 功能）	**这是"计量准"的全部。**缺任何一个，成本就归不到人头上。**没有这三个标记的调用，代码评审不通过。**
存"单价快照"而不是只存"金额"	模型价格会变。**存快照，历史成本才可复核**；只存金额，将来对不上账。
"功能标识"而不是"接口名"	老板问的是**"钱花在哪个功能上"**，不是"花在哪个接口上"。**按功能聚合才能做产品决策**（哪个功能太贵该优化）。

7.3 一条代码级纪律

> **任何调用 AI 的代码，必须带三个标记：谁调的、属于哪家公司、属于哪一类。 没有这三个标记的调用，代码评审不通过。**

为什么这条要写死在文档里：

"成本可观测"只有在"从第一行调用代码就记账"的前提下才成立——事后补是补不上的（历史数据不存在了）。

而且内测期的数据，就是单位经济模型的第一版输入——内测没记账，等于白内测。

7.4 "按消耗计费"的数据结构怎么支撑

关键设计：把"计量"和"计费"分开。

```
计量（事实层）              计费（规则层）                收款（商业层）
──────────────            ──────────────              ──────────────
ai_usage 表                计费策略表                    账单表
每一次调用一条      →      免费期 / 试用期 / 收费期   →   按周期汇总
                           配额 / 分档 / 超额处理          出账 / 对账
★ 只记事实，                ★ 全是配置，                  ★ 这一层
  不做判断                   改规则不发版                  一期不做
```

为什么这么分：

如果混在一起	分开（我们）
免费期是一套代码，收费期要重写	**免费期只记事实，收费那天加一张策略表**
改定价要发版	改配置
将来的"按 Token 计费""按席位""按功能包"都要重做	**都是策略表里的一行**

计费策略表要能表达的东西（一期只填"免费期"一行，但结构要够）：

```
计费策略 billing_policy
  id
  适用对象              公司级 / 个人级（ToB / ToC 分开，BP 要求分开核算）
  生效时间 / 失效时间
  计费方式              免费 / 按消耗 / 按席位 / 混合
  免费额度              如 每月 x 元 或 x 次
  超额度处理            拒绝 / 降级到便宜模型 / 计费 / 提醒
  分档规则              不同功能不同价（比如转写贵、对话便宜）
  状态                  启用 / 停用
```

7.5 成本怎么压住（七条机制的技术落点）

#	机制	压多少	技术上怎么落
1	**能用规则做的不用 AI**	**砍掉一整类**	代码评审必查项；"规则能算的一律走业务服务"
2	**转写改读客户已有的会议记录**	**−48%**（有前提）	优先读客户已有的转写；⚠ **商店应用能否读妙记待工单确认**
3	**知识库问答共享缓存**	**边际成本压向零**	见 7.6（**必须和"涉公司数据不缓存"划清**）

4	**分级用模型**（80% 走便宜档）	**token −63%**	按功能配默认档位；贵的档位要显式声明
5	**异步化**	同样服务器扛更多人	人能等的一律异步（复盘、日报、归因）
6	**出图配额制**	控住最贵的一类调用	按人/按月限额度；额度优先给封面
7	**成本可观测 + 报警**	防止悄悄超支	见 7.7

7.6 ★ 共享缓存：把边际成本压向零（但有一条红线）

为什么共享能成立：

```
10 万个主播问"怎么接话"  →  答案是同一个（那是行业知识，不是他的数据）
                              ↓
                    答一次，全平台复用 → 边际成本趋近于零
```

分两类，界限必须清楚：

问题类型	例子	能不能共享
涉及企业数据的	"我带多少主播？谁在下滑？"	✕**只能按公司算**（每家数据不同），**绝不进共享缓存**
不涉及企业数据的	"PK 怎么拉票？""这话能说吗？"	✅**全平台共享**

> ⚠ **一条必须写死的边界**： **共享缓存只放"行业知识类"的回答；涉及任何企业数据的回答，绝对不进共享缓存。 否则就违反了"不跨客户混用"的红线——一次违规，前面积累的信任全没。 技术上怎么保证**：缓存写入前做一次**判定**（这条回答的生成过程有没有读到企业数据）—— **有 → 不写缓存**。这个判定要做成"默认不缓存"，**只有显式标记为"行业知识类"的才能缓存。**

7.7 成本必须"看得见"，而且要"会喊"

要能看到的四个粒度：

```
全平台     这个月花了多少
  ↓
每家公司   这家客户花了多少     → 判断这个客户值不值 / 何时开始收费
  ↓
每个人     这个人花了多少       → 配额
  ↓
每一类     对话/生成/出图/转写  各花了多少 → 知道该优化哪个
```

报警三条：

报警	触发
某公司日成本异常	超过它历史均值的 N 倍

全平台日成本异常	超过预算线
单人日用量异常	明显不正常（可能是滥用或死循环）

> **BP 立了规矩：「先算清单位经济模型再规模化」。 而"算清"的前提是"看得见"。看不见的成本，永远算不出单位经济模型。**

7.8 影响什么商业结果

做对了	**做错了**
知道"每用户每月 2.2 还是 28.6 元" → **800 万撑 18 个月**	滑一档差 3-4 倍，滑两档差 13 倍 → **半年就没了**
收费那天只加一张策略表	收费要重写代码 → **错过时间窗**
能说清"哪个功能太贵"	只能整体缩水 → **把好功能也砍了**

八、AI 四档怎么走（分级用模型）

8.1 四档与模型档位

档	**谁在动**	**例子**	**默认模型档**
一 · 对话	人问，AI 答	"我现在带多少主播？"	**便宜档**（80% 走这里）
二 · 生成	AI 出一份东西，人用	复盘报告、直播脚本、培训 SOP	**主力档**
三 · 建议	AI 给方案，人选	掉人归因、下场怎么改	**主力档**
四 · 执行	AI 直接做，不问	**✕一期不做任何"不问了就做"的事**	——

> **第四档一期明确不做：AI 只给依据，人做判断**（产品铁律）。 **技术上要留出这个档位的位置**（场景类型的枚举里要有它），但**不给它开任何自动执行的入口。**

8.2 分级的落法

```
每个"功能标识"在配置里挂一个默认档位
        │
        ├─ 简单问答、字段解释、话术库命中     → 便宜档（占调用量的 80%）
        ├─ 复盘生成、脚本生成、归因分析       → 主力档
        └─ 需要显式声明才能用最贵的档
        │
```

▼
命中共享缓存 → 直接返回，一次模型都不调

档位之间差 20-30 倍（比如便宜档输入 0.2 元/百万 token vs 主力档 6.00 元）—— **所以"分级用模型"是最大的一个省钱杠杆**，也是 AI 工程这个岗位存在的理由。

8.3 AI 层三条实现原则

#	原则	为什么
1	**要能换模型**（不要绑死一家）	价格会变、能力会变；**绑定一家 = 把命交给别人**
2	**必须超时 + 降级 + 重试**	模型服务会抖；**人要能干活，不能被 AI 卡住**
3	**AI 的回答必须结构化**	一段文字要读，**结构化的一眼看完**（设计规范 5.12）

九、底座选型

9.1 先定约束（选型由 IT 团队定，我只定"必须满足什么"）

#	约束	违反了会怎样
1	**一套代码出手机和电脑**	做两套 → 维护成本翻倍、两边不一致
2	**能读能写客户的飞书表**	做不到 → 冷启动没数据（这是我们最核心的策略）
3	**多租户隔离能做到"双保险"**	做不到 → 客户不敢用
4	**AI 调用能按公司/按人记账**	做不到 → 单位经济模型算不出来
5	**岗位、职级、字段标签是配置**	写死 → 每个客户都要改代码

9.2 倾向（供参考，可推翻）

项	倾向	理由
前端	一套代码/响应式 Web，**手机优先**	满足约束 1；旧版用 Flutter 的经验可参考
后端	**用团队熟的**（旧版是 Node.js）	**"团队熟"比"最先进"重要**
数据库	**关系型 + 行级安全（RLS）**	约束 3 靠它；结算和状态流转需要事务

缓存	内存缓存（共享缓存 + 会话 + 配额计数）	7.6 靠它
向量检索	一期先用**关键词检索**，等知识库真的大了再上向量	不为了"先进"增加一期复杂度
AI	**官方直连为主 + 备一家**	官方直连明显比托管便宜；且必须能换
部署	云服务器 + 容器；**逻辑隔离用行级安全，物理隔离用命名空间**	4.3
底座	**成熟开源（多租户/RAG 能力强的）+ 自研行业层**	见 9.3

9.3 关于"用成熟开源底座 + 自研行业层"

调研到的业界通行做法（企业级 AI Agent 平台的标准形态）：

层	业界怎么做	明镜的选择
多租户 / 权限 / Agent 运行时 / RAG	**用成熟开源**（原生多租户、RAG 强、支持工具调用协议）	✅**直接用**，不重复造
行业知识 + 岗位 AI + 公会流程	没有现成的	**★ 自研——这才是我们的壁垒**

> **一句话**：**底座用买的，行业层自己写。 买底座省的是 6 个月；自研行业层攒的是"别人买不到的东西"。** ⚠ **但有一条纪律**：**底座要能换**——不能把核心数据结构和业务流程绑死在一个开源项目上。

9.4 不做清单（技术层面）

✕不做	为什么
微服务（一期）	团队小、负载低（很多人做很少的事）；**微服务会把开发速度拖慢一半。**单体 + 清晰分层够用
自建模型 / 微调	成本极高，且**行业数据还没攒够**
自建向量检索（一期）	先用关键词；等知识库大了再说
多平台同时对接（企微/钉钉）	第一阶段只做飞书；**为了"支持三个平台"把第一版做复杂是自伤**
自建 IM / 自建推送	客户已经在飞书/微信里；**通知走客户熟悉的通道**
实时音视频 / 弹幕处理	拿不到平台数据（四条路查证过）
AI 生视频	成本：30 秒 ≈ 6.9 元；10 万用户每人每月一条 = **69 万/月**
App 的推送通道适配	一期不做 App

深色模式 / 换肤	做就得两套颜色全测
自动执行档（第四档）	**AI 只给依据，人做判断**

9.5 团队要几个人

角色	人	为什么
前端	2-3	一期清单 40 个功能点对应的页面 + 两套屏（手机/电脑）
后端	2-3	约 150 个接口 + 多租户 + 权限 + **控制平面（新增）**
AI 工程	1	四档通道、分级模型、共享缓存、**成本记账**
运维/安全	0.5-1	多租户安全、备份、监控

> ★ **AI 工程这个角色不能省**：**成本控制的七条机制全靠这一块**—— 它不是"接个 API"，是"**设计怎么花钱**"。

9.6 旧版代码能复用多少（尽调会问，必须诚实）

部分	能复用吗	为什么
前端代码	✕基本不能	产品形态变了（旧版是"一个公司的后台"，新版是"平台 + 四道门"）
后端代码	✕基本不能	旧版没有数据库（本地 JSON）、没有多租户、**没有控制平面**
AI 接入经验	☑能	主备切换、超时处理、各家接口的坑——**真金白银踩出来的**
踩过的坑	☑能，而且很值钱	"别在代码里写死角色""Dev 别信任旧代码"……

> **一句话**：**旧版的价值是"踩过的坑"，不是"能用的代码"。 含糊比说清风险大得多。**

十、给 IT 的验收标准（★ 新来的后端只给这份东西、不问人就能开工）

10.1 这份文档给了什么、没给什么

给了（够开工）	没给（你们定）

五层架构与每层的职责边界	用什么语言/框架/中间件
三类账号、多身份、四道门的实现方式	具体表结构（字段类型、索引）
权限三层怎么落（公司/范围/字段）	ORM、编码规范、分支策略
多租户双保险 + 那条 CI 测试要测什么	部署拓扑、CI/CD 流水线
数据实体清单与四条设计原则	具体接口的入参出参
接口四类与 `/org` `/me` `/platform` 分门规范	缓存的具体过期策略
AI 调用记录表的字段与三个标记	监控告警的具体阈值
七条成本机制的技术落点	各模型的具体路由表
底座的五条约束	选哪个开源底座

10.2 ★ 开工验收：十条可判定的自检（回答"是/否"，不能是"大概是"）

#	自检	判定标准
1	跨公司查询**默认**被收窄了吗？	**故意不写公司条件**，查询必须返回 0 条
2	权限是在**一处**判的吗？	150 个接口里，**没有任何一个接口在自己的业务代码里写"这个人能不能看"**
3	主播身份能调到公会接口吗？	**不能。**CI 里有一条测试遍历所有 `/org/*` 用主播身份调用，断言全被拒
4	每次 AI 调用带三个标记吗？	`ai_usage` 表里，**不存在任何一条缺公司/缺人/缺功能标识的记录**
5	共享缓存会缓存企业数据吗？	**不会。**缓存写入前有判定；**默认不缓存**，只有显式标"行业知识类"才写
6	字段标签是配置还是代码？	改一个字段的可见岗位，**不需要改代码、不需要发版**
7	加一家新客户要多久？	**后台点一下完成开通**（含装入行业默认岗位模板），**不需要开发介入**
8	手机版有"电脑独有功能"吗？	**没有。**所有功能在手机网页上都能用
9	AI 的权限跟人走吗？	用低权限账号问 AI"公司里谁工资最高"，**AI 答不出来**
10	成本能按四个粒度看吗？	**全平台 / 每家公司 / 每个人 / 每一类**都能查出来，且有异常报警

这十条就是"平台版技术上立住了"的定义。 任何一条答"否"，都说明某一层被绕过去了。

10.3 ★ 开工顺序（我建议的顺序，IT 团队可调）

第一步 · 把"平台的地基"打好（不是先做业务）

① 账号 / 身份 / 任职 三张表 + 统一登录网关
② 多租户中间件 + 行级安全 + 那条 CI 测试
③ 权限三层（含字段标签）
④ ai_usage 计量表 + 三个标记的强制规范
★ 为什么先做这四件：它们是"四个门、150 个接口、七条成本机制"的地基。
先做业务的代价是—业务做完发现权限模型不对，全部返工。

第二步 · 控制平面（平台方那一侧）
入驻申请 → 审核 → 一键开通（含行业默认模板）→ 激活指标

第三步 · 公会端的骨架
主播档案 + 七段流水线 + 告警 + 跟播记录（4 步 5 字段）+ 自动复盘

第四步 · 主播端骨架（门先留好，内容可后补）
账号 + 0 必填字段 + 问一句 + 方案

第五步 · 按清单扩页面

★ 第一步的纪律："地基四件"没做完，不许开业务接口。 这是唯一一条我要求 IT 团队必须遵守的顺序——因为它决定了后面 150 个接口是"天生安全"还是"事后补丁"。

十一、与另外两份交付物的一致性核对

车总要求把这一份和 `1_产品方案.md`、`2_开发原型.html` 对一遍。**这是核对结果。**

#	产品方案 / 原型里的说法	技术方案里的落法	一致?
1	**四道门**（登录页三入口 + 申请入驻）	**三类账号 + 统一登录网关**；`/org` `/me` `/platform` `/pub` 四类路径	✅
2	公会门内**三种视野**（一线/组长/管理层），原型上是切换按钮	**靠范围规则（本人/本组/本部门/全公司）**，同一套界面	✅
3	主播端**看不到评级/去留/保底**	**四道防线**（分路 + 字段白名单 + 行级安全 + CI 测试）	✅
4	平台方**只看"体温"不看"病历"**	`/platform/*` 接口**只出汇总，不出明细内容**；脱敏在接口层	✅
5	平台方**成本页**（人均 8.4 元、按公会分摊）	`ai_usage` 表 + 四个粒度视图 + 三条报警	✅

6	平台方**激活率**（≠注册，首周 3 角色各 5 次真实动作）	激活指标表：由 `ai_usage` + 业务操作日志**按规则算出来，不存结论**（原则 5.2-4）	✅
7	**不抽佣，只收服务费**；按 Token 消耗计费	**计量与计费分离**；计费策略表（一期只填"免费期"一行）	✅
8	**不造硬件**；"一键复用参数"是给到人不是写进设备	技术方案里**没有任何硬件对接**；无设备控制接口	✅
9	**一套代码四个入口**（不是四套系统）	门户层一套；门由身份决定（2.1）	✅
10	**手机版全部功能都能用**	约束 1：一套代码出手机和电脑	✅
11	**能用规则做的不用 AI**（成本第一杠杆）	第 1 条机制 + 代码评审必查项	✅
12	知识库**共享缓存**（行业知识才共享）	7.6：默认不缓存，显式标记才能缓存	✅
13	**一期不做 App / 小程序 / 微服务**	不做清单（9.4）	✅
14	**一线只填 5 个数字以内**（跟播记录 4 步 5 字段）	跟播记录表按"只存事实"设计；**能算出来的不存**（数据原则 A 通用约定 1）	✅
15	产品方案写死的**激活定义**（首周 3 角色各 5 次真实动作）	`tenant.activated_at` + `onboarding_application.activated_at`（**存"时间点"这个事实，不存"激活"这个结论**）	✅
16	**平台方看不到客户经营数据**	`/platform/*` 走**单独的接口与查询**，不是在通用查询里开一个"不带 tenant_id"的开关（A.6 第 3 条）	✅
17	权限**字段级**（财务看保底、看不到微信电话）	`field_tag`（三种模式）+ `tenant_field_policy`（客户可改）；**`anchor_visible` 默认 false**	✅
18	线索字段：来源端口（7 个值）/ 联系电话 / 微信号	`anchor` 表按线索来源设计；**★ 联系方式在库里是 `phone` 和 `wechat` 两个独立字段**	⚠ **产品方案的表单上这两项写在一处——库里必须拆开，因为权限规则不同（见下）**

★ 第 18 条是这一轮核对新发现的、唯一一处需要写清的：

产品方案在讲"线索字段"时，把**联系电话**和**微信号**写在一格里；**但库里必须是两个字段。**原因：

字段	谁会需要	为什么不能合成一个
`phone` 联系电话	经纪人 + 负责运营 + 管理层	合规上属于**个人信息**，要按字段管
`wechat` 微信号	**同上**，但**用途不同**（加人靠它）	**两个字段将来可能给不同的人**（比如客服岗只要能打电话，不给微信）

> **影响什么商业结果**：合成一个字段的结果是——**要开就一起开，要关就一起关。 将来只要出现一次"某岗位该看微信不该看电话"（或者反过来），就得改表结构**—— **而表结构一改，历史数据要迁移，权限配置要重来。**

累计发现的三处需要补进产品方案的说法（不改结论，只是补精确）：

#	产品方案现在的说法	建议补一句	状态
1	第十节"四道门 × 六个视野"表里写"平台方门 Tab 4 个"	补一句："**平台方门多出的那个 Tab 是"成本"**——它对应 `ai_usage` 计量，是"算力中转站"的仪表盘。"	✅已补（第 3 轮）
2	2.3 节"每道门不做什么"	补一句："**主播端用独立接口路径**——不是前端不显示，是后端根本不返回。"	✅已补（第 3 轮）
3	6.3 节"线索字段"把**联系电话**和**微信号**写在一格里	补一句："**库里是两个独立字段**（`phone` / `wechat`）——因为权限规则不同，将来可能给不同的人看。"	✅已补（第 5 轮）

十二、对抗性审查

IT 负责人

#	问题	回答
1	**"这套文档够我开工吗？"**	**够到"知道做什么"这一层**，而且第十节给了十条可判定的验收自检。**具体表结构、接口定义、技术选型是你们定的。**

2	**"控制平面是新东西，会不会拖慢一期？"**	**不做它才是拖慢**：1,000 家客户靠人肉开通，商务和客户成功的人力会失控。而且它管的事（租户/权限/计量/开通）**每一件都是"必须只在一个地方做"的事。**
3	**"成本看着可控，真跑起来会不会超？"**	有这个风险，所以第 7 条机制是"成本可观测 + 报警"**。看不到的成本一定会超。** 另外：**开测第一天就要记账**，不要等上线。
4	**"客户要加字段、改流程，代码怎么扛？"**	**岗位、职级、字段标签、计费策略全是数据**（3.3 第 1 条）**。能改的必须做成配置**——旧版教训原话：「别在代码里写死角色」。
5	**"飞书授权到底能不能成？"**	接口层面是通的（多维表格 API 读写齐全）；**但 ISV 入驻门槛高，第一阶段走"一家一家配"的自建应用。** ⚠ **"商店应用能否读妙记转写"这一条还没确认，要提工单。**

未来的技术合伙人

#	问题	回答
1	**"这个架构能撑到 10 万用户吗？"**	**能。**明镜的负载是"很多人做很少的事"（一天几个操作），**不是高频并发。**真正的压力在 AI 调用——**那个靠异步 + 分级 + 共享缓存解决。**
2	**"技术债在哪？"**	**三处，都是想清楚了才欠的**：① 一期不做微服务 ② 知识库先用关键词检索 ③ **控制平面和业务在同一单体里**（将来要拆）**。不是疏忽，是排期。**
3	**"旧系统那套代码能省多少事？"**	**省不了代码，但省了弯路**（9.6）**。**

投资人技术尽调

#	问题	回答
1	**"多家公司数据放一起，怎么保证不串？"**	**双保险**（第四节）：系统自动带公司身份 + 数据库自己再查一遍**。而且有一条 CI 测试专门验证"故意漏写条件时第二道拦得住"。**

2	**"你们的技术壁垒是什么？"**	**诚实说：技术本身不是壁垒**（这些技术都能买到）。**壁垒是数据和行业 know-how。** 技术的作用是"**让数据攒得住、成本扛得住、边界漏不出去**"。
3	**"成本会不会随用户数爆炸？"**	最该被问的一问。**七条机制就是回答**，其中"共享缓存"是对它的正面反击——**让最贵那类调用的边际成本趋向零。**
4	**"主播凭什么相信你们的边界？"**	**不是承诺，是机制**：数据库行级策略 + 字段白名单 + CI 测试，**三层里任何一层单独成立都不够。**
5	**"如果你们想偷看客户数据呢？"**	技术上 `platform` 身份的接口**只出汇总不出内容**（第四节 4.5 第 2 条 + 2.2 的"平台方成员"行）。**且这条写进入驻协议。**

公会老板

#	问题	回答
1	**"我的数据放在哪？谁能看？"**	逻辑隔离（默认）：同一个数据库，**行级安全强制按公司过滤。你是数据的主人，明镜是受托处理方**（签 DPA）。
2	**"我公司里的人看得到多少？"**	按**岗位 × 职级**：本人 / 本组 / 本部门 / 全公司。**这四条规则就是行业里本来就有的层级**，你也可以改。
3	**"我不想让财务以外的人看到保底金额。"**	**字段级权限**——给"保底金额"贴一个"点名"标签，只有财务和管理层能看到。**而且这是配置，你自己能改。**
4	**"你们会不会拿我的数据去服务别人？"**	**不会。**两条硬边界：① **不跨客户混用** ② **共享缓存只放行业知识，涉企业数据绝不进**（7.6）。

主播

#	问题	回答
1	**"你怎么证明我公司看不到我在明镜里问的东西？"**	你的对话在你自己的路径（`/me/*`）下；**公司看到的是"活跃人数、用了**

		多少"，看不到内容（和平台方看公司的规则一样）。
2	**"要填多少东西？"**	**0 个必填字段。** 第一次打开看到的是方案，不是空白页。

十三、我已拍板的事项（车总知情即可）

#	决定	为什么这么定	影响什么商业结果
1	**四道门 = 一套系统 + 三类账号 + 多身份**，不做四套	门是身份决定的，不是代码分支	维护成本不翻四倍；加门不发版
2	**控制平面单独立一层**（平台级的事不写进业务）	有了 1,000 家客户，"租户/权限/计量/开通"必须是平台能力	加一家客户 = 后台点一下
3	**权限只在数据出口判一次**	150 个接口漏一个就是漏洞	权限做一整套被绕过 = 白做
4	**主播端走独立接口路径**（`/me/*`），不复用公会接口	**安全设计的默认值必须是"看不见"**	主播敢用 → ToC 成立
5	**多租户双保险 + 一条 CI 测试**	"看起来有两道"和"犯错也漏不出去"是两个估值	尽调必答；一次串号 = 1,000 家流失
6	**每次 AI 调用带三个标记，代码评审不通过就没法合**	成本可观测只有在"第一行代码就记账"时成立	800 万撑多久取决于此
7	**计量与计费分离**（免费期只记事实）	收费那天只加一张策略表	不返工、不错过时间窗
8	**共享缓存默认不缓存**，只有显式标"行业知识类"才缓存	默认值必须是安全的那一侧	不违反"不跨客户混用"红线
9	**底座用成熟开源 + 自研行业层，但底座要能换**	买底座省 6 个月；行业层才是壁垒	不把命交给一个开源项目
10	**"地基四件"没做完不许开业务接口**	先做业务 → 做完发现权限模型不对 → 全部返工	省下的是最多的时间

十四、这一轮没解决的问题（不藏）

#	没解决	归到哪
1	**飞书"商店应用能否读妙记转写"** —— 直接影响成本压缩机制第 2 条（−48%）	**要提工单**；车总沟通时也可顺带问
2	企微 / 钉钉的开放平台机制	第二阶段的事，一期只做飞书
3	具体选哪个开源底座	IT 团队选型；**我只定"要能换"这条约束**
4	~~具体表结构、索引~~ → **已补：见附录 A**	分库分表仍归 IT（一期不需要，见 A.8）
5	各模型的**具体路由表**（哪个功能走哪个档）	AI 工程岗；**内测期边跑边定**
6	监控告警的具体阈值	运维；**先定"要有"，阈值靠运行数据调**
7	物理隔离（大客户）的具体实现	一期不做，但**数据结构已留位置**（每表带公司列）

附录 A • 表结构建议稿

> **这一份要满足一个验收标准**：**新来的后端只给这份东西，能照着把库建起来。 写法约定**： - **每一节开头一句大白话**（这事是干什么的、影响什么商业结果） - 字段表里给**类型 / 可空 / 默认 / 说明** - **重要的设计后面写"为什么"** —— 尤其是那些"换个做法也能跑，但会出事"的地方 - SQL 风格示意（PostgreSQL 语法），**具体写法 IT 团队可以调，约束不能丢**

A.0 先定十条通用约定

> **这一节比后面所有表都重要。** 这十条定了，**一期清单里那 40 个功能点**（原型已覆盖 15 个画面）的数据结构就不会乱。 **口径说明**（第 8 轮全局收口时统一）：产品方案第十节的一期清单是 **40 个功能点**（P 9 + M 6 + E 17 + C 8）。 本文档早期写的"45 个页面"是**照着做的规模感**，不是一期承诺——**已改成一期清单的口径。**

#	约定	为什么（影响什么）

1	**主键一律 `bigint`（自增/雪花），不用 UUID 当主键**	UUID 随机写入会打散索引，**表一大就慢**。而且 UUID 会被遍历
2	**对外的 ID 用 `public_id`（短随机串，唯一）**	★ **防遍历**：不能让用户把 URL 里的 `123` 改成 `124` 就看到别家数据。**这是"看不见"的第一道保险**
3	**每张业务表都带 `tenant_id`，且索引的第一列永远是它**	① 多租户隔离（行级安全靠它）② **所有查询都带公司条件，复合索引必须从它开头**
4	**一张表不设数据库外键约束**（唯一例外见下），只建索引	高写入量下的外键检查会拖慢；**但跨租户的关联必须由应用层保证**（A.6 第 3 条）
5	**例外：`ai_usage` 的 `(tenant_id, identity_id)` 用复合外键**指向 `identity(tenant_id, id)`	★ **成本归集不能错。**这条约束保证"一条用量记录不可能挂到别家公司的人身上"
6	**金额一律 `numeric(12,6)`**（元），**不用浮点数**	浮点数算钱会出现 0.1+0.2≠0.3 这种事，**账单对不上**
7	**时间一律 `timestamptz`**	跨时区不出错（服务器、客户、我们可能都不在一个时区）
8	**软删除**：`deleted_at timestamptz null`，**不做物理删除**	合规留痕；而且**删了的主播在历史漏斗里还要算**
9	**所有表都有 `created_at / updated_at`**	基础审计
10	**枚举值用 `varchar(32)` + 注释，不用数据库 enum 类型**	enum 加值要改表结构（锁表）；**客户要能自己配岗位和字段，枚举会长**

再补一条给写代码的人：

> **任何一条查询，如果 `tenant_id` 不是从登录态来的，而是从前端参数来的——代码评审不通过。**（`ai_usage` 和高层的平台方查询是唯一例外，它们要跨租户——但只对 `platform` 身份开放。）

A.1 地基四张表：租户 / 账号 / 身份 / 任职

> **大白话**：这四张表回答"**谁、在哪家公司、干什么岗、能看多远**"。 **它们是"四道门"的地基**——门不是页面，是这四张表查出来的结果。

① `tenant` —— 一家公会 = 一个租户

字段	类型	可空	默认	说明
`id`	bigint	否	自增	主键
`public_id`	varchar(24)	否		对外 ID（约定 2）
`name`	varchar(128)	否		公司名（公会名）
`short_name`	varchar(32)	是		简称（界面上显示用）
`logo_url`	varchar(255)	是		客户自己的 logo（**只显示 logo，配色不动**——设计规范 8.3）
`type`	varchar(32)	否	`'guild'`	`guild` 公会 / **没有商家**（车总明确：明镜没有商家，只有公会）
`status`	varchar(32)	否	`'applying'`	`applying` 申请中 / `active` 已开通 / `suspended` 停用 / `closed` 关闭
`scale_staff`	smallint	是		员工规模（入驻时填的，用来推荐岗位模板）
`scale_anchor`	smallint	是		线下主播规模
`isolation_mode`	varchar(16)	否	`'logical'`	`logical` 逻辑隔离 / `physical` 物理隔离（**一期只用 logical，字段先留着**）
`activated_at`	timestamptz	是		★ **激活时间**（见下面的"为什么"）
`dpa_signed_at`	timestamptz	是		《数据处理协议》签订时间（合规台账）
`created_at` / `updated_at` / `deleted_at`	timestamptz			通用约定 8、9

★ 为什么单独存一个 `activated_at`：

"入驻完成" ≠ "注册了"，= **开户后首周至少 3 个角色各做过 5 次以上真实动作**（产品方案 4.3 写死的定义）。

这个字段是"1,000 家里多少在真用"的唯一答案。 存它，是为了不必每次去重算一遍（也为了不给"事后改口径"留空间）。

平台方后台的"激活率"，就是数这个字段非空的租户 ÷ 全部租户。

② `user` —— 账号（一个人只有一个）

字段	类型	可空	默认	说明
`id`	bigint	否		主键
`public_id`	varchar(24)	否		对外 ID
`phone`	varchar(32)	否		手机号，**唯一**（登录用）
`password_hash`	varchar(128)	是		手机验证码登录时为空
`name`	varchar(64)	是		姓名（用户自己填的）
`avatar_url`	varchar(255)	是		
`status`	varchar(32)	否	`'active'`	`active` / `disabled` / `closed`
`last_login_at`	timestamptz	是		用来算"沉默"
`created_at` / `updated_at` / `deleted_at`	timestamptz			

★ 为什么账号表里没有 `tenant_id`：

一个人可以同时是两个身份——比如他既是某公会的运营，自己也在播。

账号是"人"，不是"公司的人"。 把 `tenant_id` 放在账号表上，第一个真实案例就会撞上。

> **影响什么商业结果**：如果写死"账号属于哪家公司"，**行业里最常见的"运营兼主播"就没法用一个账号**—— 他要注册两个号，数据割裂，**ToC 和 ToB 两条线就串不起来了**（而 ToC 的获客管道正是 ToB）。

---

③ `identity` —— 身份（一个人可以有多个）★ 全系统的中枢

字段	类型	可空	默认	说明
`id`	bigint	否		主键

`tenant_id`	bigint	**是**		归属公司。**`member`/`anchor` 可空**（见下）
`user_id`	bigint	否		→ `user.id`
`kind`	varchar(16)	否		★ `member` 公会成员 / `anchor` 主播 / `platform` 平台方成员
`status`	varchar(16)	否	`'active'`	`active` / `disabled`
`created_at` / `updated_at`	timestamptz			

唯一约束：`unique (user_id, kind, tenant_id)` —— 防同一个人在同一家公司重复建身份。

★ 为什么 `tenant_id` 可空，以及三种身份各自的约束：

kind	tenant_id	为什么
`member`	**必填**	公会成员一定属于某家公司
`anchor`	**可空**	**★ 外部主播（没有公会）必须能独立注册——野生主播是 ToC 的真实客户**（产品方案 7.5）。**把主播绑死在公司上，等于把 ToC 的天花板砍掉**
`platform`	**必须为空**	平台方不属于任何客户公司。**留一个 `tenant_id` 非空的 platform 身份，就是留了一个越权的口子**

④ `employment` —— 任职（岗位 + 职级 = 看多远）

字段	类型	可空	默认	说明
`id`	bigint	否		主键
`tenant_id`	bigint	否		**必须与 `identity.tenant_id` 一致**（见下面的约束）
`identity_id`	bigint	否		→ `identity.id`
`department_id`	bigint	是		→ `department.id`
`role_id`	bigint	否		→ `role.id`（决定"能看哪些功能"）

`rank`	varchar(16)	否	`'member'`	**★ `member` 本人 / `lead` 本组 / `director` 本部门 / `exec` 全公司**
`started_on`	date	否		生效日
`ended_on`	date	是		失效日。**空 = 在职**
`created_at` / `updated_at`	timestamptz			

约束：`unique (tenant_id, identity_id) where ended_on is null`（一个人在一家公司同时只能有一个"在职任职"）

约束：`check (rank in ('member','lead','director','exec'))`

★ 为什么职级只有四个值：

权限的"范围"层就靠这四个值（产品方案 3.3 / 技术方案 3.2）：**本人 / 本组 / 本部门 / 全公司。**

只有四条规则，所以不用培训、不用文档、不会记错。 客户能改的是"谁是什么职级"，不是"有几种职级"。

★ 为什么离职不删记录，而是写 `ended_on`：

① **合规留痕**（法规要求的四件事里有"约束表演者行为"，出纠纷要能说明"当时是谁在管"）

② **掉人归因要算历史**——如果离职就把记录删了，**三个月前的漏斗数据就变了。**

A.2 权限三张表：角色 / 权限 / 字段标签

> **大白话**：这三张表回答"**这个岗位能点哪些按钮、能看哪些列**"。 **★ 关键：这些是"数据"，不是"代码"**——改配置不发版（旧版教训原话：别在代码里写死角色）。

⑤ `role` —— 岗位（系统预置 11 个 + 客户可自定义）

字段	类型	可空	默认	说明
`id`	bigint	否		主键
`tenant_id`	bigint	**是**		**★ 空 = 系统预置（行业默认）；有值 = 这家客户自己加**

				的
code	varchar(48)	否		**稳定标识**（如 ops、agent、finance）
name	varchar(64)	否		显示名（客户可以改名）
dept_kind	varchar(32)	是		归哪条线：ops / agent / visual / video / admin / finance / mgmt
default_rank	varchar(16)	否	'member'	新建这个人时默认职级
is_system	boolean	否	false	系统预置的不能被删，只能改名
sort	smallint	否	0	界面排序
created_at / updated_at	timestamptz			

唯一约束：`unique (tenant_id, code)`

★ 为什么 `tenant_id` 可空（而不是"每家复制一份"）：

新公会开号 → 系统把 11 个预置岗位【复制】到这家公司 → 它只改名字和人数

这是"开箱即用"的关键：新客户看到的是一个**已经装好岗位的系统**，不是空白系统。

产品方案 4.3 写过：新公会开号看到空系统，80% 的人在这里就走了。

做法	结果
✕只有一份全局岗位表	客户不能加自己的岗位 → 每加一个岗位要改代码
✕完全靠客户自己建	开号就是空的 → **80% 流失在第一步**
☑**预置模板 + 开号时复制**（选择）	开箱即用，且客户能改

⑥ permission + role_permission —— 功能权限

permission 字段	类型	说明
id	bigint	
code	varchar(64)	**权限码**，全局唯一（不分公司）
name	varchar(64)	显示名
group_name	varchar(32)	界面分组（主播 / 记录 / 结算 / 设

		置...)

role_permission 字段	类型	说明
role_id	bigint	
permission_id	bigint	
can_write	boolean	false = **只看**（对应设计规范里的"只看"）

权限码命名规范（给 IT 的硬约定）：

```
<对象>.<动作>              如 anchor.edit / followup.create
<对象>.<敏感字段>.view      如 anchor.salary.view（保底金额）/ contact.view（联系方式）
```

★ 为什么权限码要"点号分段"：

字段级权限（`anchor.salary.view`）和功能权限用同一套编码——

这样"能不能看保底金额"和"能不能改主播档案"在代码里是同一种东西，

权限判断只有一处（技术方案 3.1），不会出现"功能权限管了、字段权限忘了"。

⑦ field_tag + tenant_field_policy —— 字段级权限

> **大白话：同一张表，不同的人看到不同的列。** 这是"财务第一次敢把工资数据放进系统"的唯一前提。

field_tag 字段	类型	说明
id	bigint	
code	varchar(48)	**系统固定**，如 anchor.phone（联系电话）、anchor.salary（保底金额）、anchor.rating（评级）
label	varchar(64)	显示名，如"保底金额"
default_mode	varchar(16)	★ public 公开 / scope 按范围 / named 点名
anchor_visible	boolean	**★ 主播本人能不能看到自己这个字段**（默认 false——见下面的"★"）

tenant_field_policy 字段	类型	说明
tenant_id	bigint	
field_code	varchar(48)	

`mode`	varchar(16)	覆盖默认（客户可以改：A 公司觉得流水敏感，B 公司觉得年龄敏感）
`allowed_role_codes`	text[]	`named` 模式下，哪些岗位能看

★ 一个必须写死的默认值：

> **`field_tag.anchor_visible` 的默认值是 `false`。 意思是：任何一个新加的字段，主播默认看不到。 要让它对主播可见，必须显式打开。**

为什么默认是"看不见"：安全设计的默认值必须是收窄的那一侧。

如果默认可见，那么"某天某人加了一个新字段、忘了配"的结果就是**主播看到了不该看的东西**——**而这一类事故是不可逆的**（看到了就是看到了）。反过来，默认不可见时忘了配，**最坏结果是主播少看到一个字段**，业务还能跑。

> **影响什么商业结果：主播端的三条硬纪律（0 必填 / 看不到评级 / 看不到去留）里，后两条就靠这个默认值兜底。 不靠"记得配"，靠"不配就看不见"。**

---

⑧ `field_whitelist` —— 主播端接口的字段白名单

字段	类型	说明
`id`	bigint	
`api_scope`	varchar(32)	目前只有 `me`（主播端）
`object_name`	varchar(48)	如 `anchor`、`session`（场次）
`field_name`	varchar(48)	允许返回的字段名
`note`	varchar(128)	为什么放它进来

★ 这张表是"四道防线"的第二道（技术方案 2.5）：

```
主播端接口 /me/* 返回某个对象时：
  遍历字段 → 不在白名单里的【直接丢弃】
  ★ 默认全禁，显式放行
```

它的价值在于"可被测试"：CI 里那条测试断言的正是

"用主播身份调 /me/*，返回的 JSON 里的每个字段名都能在这张表里找到"。

没有这张表，"字段不泄露"就无法自动验证。

A.3 业务核心三张表：主播 / 流水线事件 / 入驻申请

⑨ anchor —— 主播档案（全系统的中心）

> **为什么主播是中心**：**七段流水线、11 个交接点，最后都汇到"一个人身上"**—— "一页看完一个主播"这个产品要求，决定了数据结构必须以他为中心。

字段	类型	可空	说明
id	bigint	否	
public_id	varchar(24)	否	对外 ID
tenant_id	bigint	否	归属公司
identity_id	bigint	是	★ 主播注册了明镜账号才有（**外部主播也有**）
name	varchar(64)	否	姓名
nickname	varchar(64)	是	直播昵称
gender	varchar(8)	是	
age	smallint	是	
phone	varchar(32)	是	★ 敏感字段 anchor.phone（点名可见）
wechat	varchar(64)	是	★ 敏感字段
douyin_no	varchar(64)	是	抖音号（平台侧标识）
track	varchar(32)	是	赛道（才艺唱歌 / 颜值聊天 / 舞蹈…）
persona	varchar(255)	是	人设（定人设那一段的产出）
level	varchar(8)	是	等级 C/B/A/S（**注意：评级走 `field_tag`，不给主播看**）
guarantee_amount	numeric(12,6)	是	★ 保底金额（敏感字段 anchor.salary，**点名可见**）
sign_status	varchar(16)	是	签约状态
signed_on	date	是	签约日
contract_end_on	date	是	★ **合同到期日**（"签约 30 天内到期"预警靠它）
stage	varchar(16)	否	★ **当前所在段**：sourcing/interview/training/persona/styling/schedule/live/review（七段流水线）
attrition_stage	varchar(16)	是	★ **在哪一段掉的**（anchors.stage 的历史值，给人看）
attrition_reason	varchar(255)	是	流失原因
status	varchar(16)	否	active 在册 / churned 已流失 / paused 暂停
owner_identity_id	bigint	是	★ **负责运营**（"本人"范围规则就靠它）

`group_id`	bigint	是	★ **所属组**（"本组"范围规则靠它）
`dept_id`	bigint	是	★ **所属部门**（"本部门"范围规则靠它）
`created_at` / `updated_at` / `deleted_at`	timestamptz		

★ 为什么 `owner_identity_id` / `group_id` / `dept_id` 要冗余存在主播表上：

权限的范围规则（本人 / 本组 / 本部门 / 全公司）**在每次查询时都要用。**

如果每次都去 join `employment` 反查"这个人属于哪个组"，**150 个接口每个都要多 join 一次。**

冗余这三个字段，是拿"一处可能的更新不一致"换"每条查询都快"——

而这个更新只可能发生在"转组/换负责人"这一个地方，可控。

（这也是约定 4"不设外键、靠应用层保证"的代价与收益。）

索引	为什么
`(tenant_id, status, stage)`	打开就是"在册的主播、各在哪一段"
`(tenant_id, owner_identity_id)`	一线"我带的 15 个主播"，**最高频的查询**
`(tenant_id, group_id)`	组长看本组
`(tenant_id, dept_id)`	总监看本部门
`(tenant_id, contract_end_on)`	签约到期预警
`unique (tenant_id, public_id)`	对外 ID 查

⑩ `pipeline_event` —— 流水线事件（每次状态变化一条）

> **大白话**：主播每往前走一段，就记一条。**这是"掉人归因"的数据源。**

字段	类型	说明
`id`	bigint	
`tenant_id`	bigint	
`anchor_id`	bigint	
`from_stage`	varchar(16)	从哪一段
`to_stage`	varchar(16)	到哪一段
`result`	varchar(16)	`pass` 通过 / `fail` 淘汰 / `withdraw` 自己走

gate_code	varchar(32)	过的是哪道闸门（五道闸门之一）
operator_identity_id	bigint	谁操作的（留痕）
note	varchar(500)	备注
occurred_at	timestamptz	★ 发生时间（漏斗按它算周期）

索引：(tenant_id, occurred_at)、(tenant_id, anchor_id, occurred_at)

★ 为什么"用事件流转"而不是"在主播表上改一个状态字段"：

只改状态字段	**事件流水（选择）**
只能知道"现在在哪一段"	**知道"每一段花了多久、从哪掉的多"**
改了就查不到历史	历史永远在（数据设计原则 3：原始数据不覆盖）
掉人归因做不出来	**★ "掉人归因"这个功能就是这张表撑起来的**

> **影响什么商业结果**：BP 的第二层价值"少掉人"= 多赚钱，就是这张表的产出。 "淘汰环节""流失环节"这两个字段客户已经在记了，只是没人算——这张表就是"算"的地方。

---

⑪ onboarding_application —— 入驻申请

> **大白话**：一家公会从"想用"到"真的在用"，全过程记在这。 **它是"1,000 家"这条主线的账本。**

字段	**类型**	**说明**
id	bigint	
public_id	varchar(24)	
company_name	varchar(128)	申请填的公司名
contact_name	varchar(64)	联系人
contact_phone	varchar(32)	联系手机
scale_staff	smallint	员工规模
scale_anchor	smallint	主播规模
license_url	varchar(255)	营业执照（**文件存对象存储，库里只放地址**）
guild_screenshot_url	varchar(255)	公会后台截图（核对资质用）
status	varchar(16)	★ pending 待审 / approved 通过 / rejected 驳回 / opened 已开户
reject_reason_code	varchar(32)	**★ 驳回理由用"码"，不用手写**（后台点模板，不用打字）

reviewed_by	bigint	审核人（→ user.id，明镜员工）
reviewed_at	timestamptz	
tenant_id	bigint	通过后关联到的租户
channel	varchar(32)	★ **从哪来的**：bd 商务 / community 社群 / content 内容 / referral 老带新
referrer_tenant_id	bigint	老带新的，是哪家带进来的
activated_at	timestamptz	激活时间（和 tenant.activated_at 一致，冗余一份便于漏斗统计）
created_at / updated_at	timestamptz	

索引：(status, created_at)、(channel)

★ 为什么要 `channel` 和 `referrer_tenant_id` 两个字段：

"2-3 个商务怎么铺 1,000 家"这个问题，只能靠数据回答。

哪个渠道来的公会激活率高，钱和人就往哪里投。

"老带新"如果管用（公会老板互相推荐），那是成本最低的一条路——

但只有记了 `referrer_tenant_id`，才知道它到底管不管用。

A.4 计量表：ai_usage（★ "算力中转站"的地基）

> **这一张表是整份方案里"最贵"的一张**——它决定的不是功能，是**毛利率**。

字段	类型	说明
id	bigint	
tenant_id	bigint	**★ 一号标记：哪家公司**
identity_id	bigint	**★ 二号标记：谁调的**
feature_key	varchar(48)	**★ 三号标记：哪个功能**（如 followup.recap 跟播复盘生成）
scene_kind	varchar(16)	四档：chat 对话 / gen 生成 / suggest 建议 / exec 执行（**一期 exec 永远为空**）

`model_key`	varchar(48)	用了哪个模型
`model_tier`	varchar(16)	档位：`cheap` / `main` / `premium`
`input_tokens`	int	
`output_tokens`	int	
`unit_price_in`	numeric(12,6)	★ **输入单价快照**（元/百万 token）
`unit_price_out`	numeric(12,6)	★ **输出单价快照**
`cost_amount`	numeric(12,6)	本次成本（元）
`cache_hit`	boolean	命中共享缓存（**命中时 cost_amount = 0**）
`duration_ms`	int	耗时
`result`	varchar(16)	`ok` / `fail` / `timeout`
`idem_key`	varchar(64)	★ **幂等键**（防重试导致重复计费）
`occurred_at`	timestamptz	发生时间（**分区键**）

分区：**按 `occurred_at` 做月分区（range partition）**

索引：`(tenant_id, occurred_at)`、`(tenant_id, identity_id, occurred_at)`、`(tenant_id, feature_key)`

★ 四条设计决定，每条都要写清"为什么"：

① 存"单价快照"，不存"金额"（或者说不只存金额）

> 模型价格会变，而且变得很频繁。 **只存一个 `cost_amount`，三个月后有人问"这笔账怎么算出来的"，你答不上来。 存了单价快照，历史成本永远可复核。** —— 这对"跟客户解释账单"和"尽调看账"都是必需的。

---

② `feature_key`（功能标识），不是"接口名"

> 老板问的永远是"**钱花在哪个功能上**"，不是"花在哪个接口上"。 **按功能聚合才能做产品决策**（哪个功能太贵该优化、该加配额）。 **按接口聚合只能得出"这个接口调用多"——这句话不构成任何决策。**

---

③ 三个标记缺一不可（`tenant_id` / `identity_id` / `feature_key`）

> **缺任何一个，成本就归不到人头上。 这是"成本可观测"的物理前提**——技术方案 7.3 把它写成了一条代码评审不通过项。

---

④ 用复合外键约束 `(tenant_id, identity_id) → identity(tenant_id, id)`

> **成本归集不能错。** 这条约束保证"一条用量记录**不可能**挂到别家公司的人身上"。 **如果算错一家客户的成本，我们对这家客户的定价就是错的。**

★ 数据量估算（给运维的）：

```
一期假设：10 万活跃用户 × 每人每月 130 次调用 = 1300 万行/月
6 个月 ≈ 7800 万行
单行约 200 字节 → 数据约 15.6 GB，加索引约 25-30 GB
```

结论：**单台关系型数据库能扛住，但必须按月分区。**

半年后按实际增长决定是否把聚合结果单独存一张汇总表（或迁到列存）——

但"明细"必须留着，因为明细是"能复核"的前提。

★★ 给运维的六条落地提示（建库时就按这个做，别等出事）：

#	提示	为什么
1	**`ai_usage` 和 `audit_log` 建表时就按 `occurred_at` 做月分区**	7800 万行如果不分区，超过半年后**按公司+时间查会越来越慢**；分区之后"查某个月"只扫一个分区
2	**索引严格按 A.4 给的三条建，第一条永远是 `tenant_id`**	这个系统**每一条查询都带公司条件**，所以复合索引必须以它开头——**顺序反了等于没建**
3	**监控"日增行数"和"单行平均大小"两个指标**	上面那个 7800 万行是**用假设用量算的。真实用量和内测数据一比就知道差多少**——这两个指标是修正单位经济模型的第一手数据
4	**磁盘容量按"估算的 3 倍"准备**	估算用的是"每人每月 130 次"，**真实用量可能高得多**（BP 里用量是假设，明确标了"待内测验证"）
5	**`audit_log` 的保留策略要早定**（建议：明细留 12 个月，之后转归档存储）	合规要求"能查"，**但没有要求"永久在线"**。归档能查、又省主库
6	**`ai_usage` 明细绝对不删**	它是**账单的原始凭证。**可以归档，不能删——**明细没了，三个月后有人问"这笔账怎么算的"就答不上来**

> **★ 为什么这几条要写给运维（而不只是给开发）：** **"800 万撑多久"这个问题，一半是成本单价，另一半是数据增长。** 单价会跌、用量会涨、数据只增不减——**运维手里的这三个数字，决定了这套系统能不能撑到 A 轮。**

A.5 审计表：audit_log

> **大白话**：谁改了什么、什么时候改的、原来是什么。 **合规要求，也是出纠纷时唯一拿得出来的东西。**

字段	类型	说明
id	bigint	
tenant_id	bigint	
actor_identity_id	bigint	谁
action	varchar(32)	create / update / delete / view_sensitive（**★ 看敏感字段也记**）
object_type	varchar(48)	什么对象（anchor / settlement…）
object_id	bigint	哪个
before_json	jsonb	改之前（**只存被改的字段**，不存整行）
after_json	jsonb	改之后
ip	varchar(64)	
occurred_at	timestamptz	

索引：(tenant_id, object_type, object_id, occurred_at)、(tenant_id, actor_identity_id, occurred_at)

分区：同 ai_usage，按月

★ 为什么 `action` 里有 `view_sensitive` 这一类：

"看"也要记。

财务看了某个主播的保底金额、总经办看了联系方式——**这些动作本身要留痕。**

理由有两个：① 合规（敏感个人信息要能追溯谁看过）② **给客户的承诺**——

我们跟公会老板说"看体温不看病历"，**那就得能证明我们确实没看。**

A.6 表之间的关系（E-R 概览）

```
tenant (公会/企业)
  │
  ├──< identity >──── user          ★ 一个人可有多个身份（运营兼主播）
  │       │  kind: member / anchor / platform
  │       │
  │       └──< employment >── role ──< role_permission >── permission
  │              (tenant_id, identity_id, role_id, rank)          │
  │                                                          (权限码)
  ├──< department
  │
  ├──< anchor ──< pipeline_event     ★ 主播是中心；事件流水撑"掉人归因"
  │       │
  │       └── identity_id  ──→ identity(kind='anchor')   ← 主播注册了账号才有
  │
  ├──< ai_usage  ★ (tenant_id, identity_id) 复合外键 → identity
  │
  ├──< audit_log
  │
  └──< onboarding_application ──(通过后)──> tenant

全局表（无 tenant_id）：permission · field_tag · field_whitelist
半全局表（tenant_id 可空）：role（空 = 系统预置模板）
```

★ 三条关系上的硬约束（写进代码评审清单）：

#	约束	为什么
1	**`employment.tenant_id` 必须等于 `identity.tenant_id`**	不然会出现"这个人的任职在 A 公司、身份在 B 公司"——**权限范围一算就串**
2	**`ai_usage.identity_id` 对应的人，其身份必须属于同一个 `tenant_id`**	用复合外键强制（A.4 ④）。**成本错一家，定价就错一家**
3	**跨租户的关联一律不允许，除了 `platform` 身份**	**平台的"看全局"能力只给 `platform`，而且只出汇总不出内容**（技术方案 2.2）

★ 第 3 条要特别说清它的实现方式：

```
平台方查询（/platform/*）：
   不是"给 platform 身份一个跨租户的 WHERE 条件"
   而是【走单独的接口 + 单独的查询 + 输出前脱敏】

   为什么：因为一旦在通用查询里允许"不带 tenant_id"，
         那么任何一个开发者写漏了条件，就变成跨租户泄露。
```

跨租户能力必须是"另一条路"，不是"同一个接口的开关"。 这是技术方案 2.1 那条原则在数据层的落地。

A.7 ★ 一次查询要走哪些表（把权限串起来看）

> **这一节是给后端看的最实用的一节**：一个接口从收到请求到给出数据，中间发生了什么。

```
① 请求进来（带会话 token）
        ↓
② 网关：验 token → 拿到 user_id
        ↓
③ 查 identity：这个人现在用的是哪个身份？
   （多个身份时由前端选择，token 里带着 identity_id）
        ↓
④ 按 kind 分流：
   ├─ member   → 查 employment → 拿到 role_id / rank / dept / group
   │              → 范围 = 按 rank 决定（本人/本组/本部门/全公司）
   │              → 功能权限 = role_permission
   │              → 字段权限 = field_tag + tenant_field_policy
   │
   ├─ anchor   → 范围 = 【只有他自己】（owner_identity_id = 我 或 identity_id = 我）
   │              → 字段 = 【field_whitelist 白名单】（默认全禁）
   │
   └─ platform → 走单独接口，出汇总，输出前脱敏

⑤ 数据出口（★ 权限只在这一处判断）：
   自动加 tenant_id 条件 → 数据库行级安全再查一遍 → 按字段标签裁列
        ↓
⑥ 返回
```

★ 这一段里最关键的一句：

> **业务代码（第 ⑤ 步之前的所有代码）不知道权限是怎么算的。 它只管"我要查什么"，不管"谁能看"。**

这就是"权限只在一处判断"的含义（技术方案 3.1）——

150 个接口里没有任何一个，在自己代码里写"这个人能不能看"。

A.8 建表顺序（★ 照着这个顺序建，不要跳）

> **理由在技术方案 10.3**：地基四件没做完不许开业务接口。 这一节是那句话的**执行清单**。

批	建什么	建完能验证什么
第 1 批	`tenant` · `user` · `identity` · `employment`	**一个人能有两个身份**（造一个"运营兼主播"的测试数据）
第 2 批	`role` · `permission` · `role_permission` · `department`	开号时能复制 11 个预置岗位
第 3 批	`field_tag` · `tenant_field_policy` · `field_whitelist`	**主播身份拿到的主播对象里**

		没有敏感字段
第 4 批	行级安全策略 + **那条 CI 测试**（故意漏写条件，断言返回 0 条）	**★ 双保险真的双**
第 5 批	`ai_usage` + 三个标记的强制规范	**每次调用都能查到"谁、哪家公司、哪个功能"**
第 6 批	`audit_log`	改一个主播档案，能查到改前改后
第 7 批	`onboarding_application`（+ 平台方那一侧）	**一家公会能从申请走到激活**
第 8 批	`anchor` · `pipeline_event`	七段流水线能跑，漏斗能算
之后	业务表（跟播记录、复盘、培训、妆造、排期、结算…）	按产品方案第十节的功能清单逐批做

★ 一条硬规矩：第 1-6 批没做完，不许开第 8 批之后任何一个业务接口。

先做业务的代价是"做完发现权限模型不对，全部返工"——这是最贵的一种返工。

A.9 分库分表与其他（一期不做，但要说清）

项	一期的做法	什么时候要动
分库	单库（多租户靠行级安全）	**一家客户的数据量到几百 GB** 才考虑（公会这个行业不容易到）
分表	单表 + **按月分区**（`ai_usage` / `audit_log`）	已有分区，够用
读写分离	一期不做	报表查询变慢时（**先加索引，别急着加机器**）
把 `ai_usage` 挪到别的地方	一期留在主库	半年后按实际增长定（**但明细要留，明细是"能复核"的前提**）
物理隔离（大客户）	不做，但 `tenant.isolation_mode` 字段已留	**来了一个"数据必须独立存"的大客户时**

A.10 附录小结：这份表结构定了之后，哪些事变成了"不可能"

> **一份好的数据结构，价值不在"能存下"，在"有些错事做不出来"。**

#	变成了不可能的事	靠什么
1	**把 A 公司的数据查给 B 公司**	每表带 `tenant_id` + 索引首列 + 行级安全（约定 3）
2	**通过改 URL 里的数字看别家数据**	对外用 `public_id`（约定 2）
3	**把一条 AI 用量记到别家公司头上**	`(tenant_id, identity_id)` 复合外键（A.4 ④）
4	**主播看到新加的敏感字段**	`anchor_visible` 默认 `false`（A.2 ⑦）
5	**主播端接口多返回一个字段**	`field_whitelist` 默认全禁 + CI 测试（A.2 ⑧）
6	**算不出某个月的钱是怎么花的**	单价快照（A.4 ①）
7	**把某个月的漏斗数据改掉**	事件流水，原始数据不覆盖（A.3 ⑩）
8	**说清"当时是谁在管这个主播"**	离职不删记录，写 `ended_on`（A.1 ④）

> **这八条就是"新来的后端照着建库"要守住的东西。 表名可以改、字段可以加、类型可以调——这八条不能丢。**

附录 B • 接口清单（给 IT 的开工包）

> **这一份给什么**：**接口的组织原则 + 清单。 不是什么**：不是 API 文档——**每个接口的入参出参，是 IT 团队自己该产出的。 验收标准**：**新来的后端拿到这份，能照着把接口骨架搭出来，而且不会把主播端和公会端混在一起。** 配套：附录 A 是表结构（数据长什么样），**本附录是接口（数据怎么进出）**。

B.0 先立一条总原则

> **接口的路径，就是"门"在代码里的样子。**

产品上的"四道门"，在代码里就是**四段路径前缀**：

门	路径前缀	身份	数据范围
（未登录）	`/pub/*`	无	只有公开数据

公会门	`/org/*`	`member`	**本公司**，按岗位×职级收窄
主播门	`/me/*`	`anchor`	**只有他自己**
平台方门	`/platform/*`	`platform`	全平台，**但只出汇总**

为什么这件事值得单独立一节：

门分错了，权限体系就白做了。

一次"把主播端的接口挂在 `/org` 下面"的偷懒，**等于把四道防线里的第一道直接拆掉**（技术方案 2.5）。

B.1 命名规范（照这个来，不许自己发明）

B.1.1 基本格式

```
/<前缀>/<资源>/<动作>                  ← 动作是"能数得出来的白名单"
/<前缀>/<资源>/<子资源>/<动作>         ← 需要时才用第三段
```

例子：

接口	意思
`GET /org/anchor/list`	公会门 · 主播 · 列表（带分页与筛选）
`POST /org/anchor/create`	公会门 · 主播 · 新建
`GET /org/anchor/detail`	公会门 · 主播 · 详情（**"一页看完"那个接口**）
`POST /org/followup/create`	公会门 · 跟播记录 · 新建
`GET /me/plan/current`	主播门 · 方案 · 当前这场

B.1.2 动作白名单（只有这 8 个）

动作	用途	幂等?
`list`	列表（**永远带分页 + 筛选**）	是
`detail`	单个对象的完整信息	是
`create`	新建	否
`update`	修改	是
`remove`	**软删除**（不物理删，见 A.0 约定	是

	8）	
`export`	导出	是
`submit`	提交（走状态流转的：推进流水线、提交表单）	否
`approve` / `reject`	审批通过 / 驳回（**只用在有闸门的地方**）	是

★ 为什么要"白名单"而不是"随便起名"：

同一个业务动作，十个人会起十个名字（`getAnchorList` / `anchorList` / `listAnchors` / `queryAnchor`...）。

白名单的作用不是"好看"，是让"这个接口干什么"在路径上一眼可读——

而这直接决定了**"权限码能不能和数据路径对应上"**（见 B.3 第 3 条）。

B.1.3 反例（这些命名是错的，为什么）

✕错的	为什么错	☑对的
`GET /org/getAnchorList`	**动词重复**（`GET` 已经是动词）	`GET /org/anchor/list`
`GET /org/anchor/queryByOwnerAndStage`	**把查询条件写进路径**——条件会变，路径不该跟着变	`GET /org/anchor/list?owner=&stage=`
`POST /org/anchor/doSomething`	**`doSomething` 不说人话**，看不出干什么	`POST /org/anchor/submit`
`GET /org/all`	**"all"是范围词，不是资源**——这种接口最容易变成越权入口	`GET /org/anchor/list`（范围由登录态决定）
`GET /me/anchor/list`	**★ 主播端拿"主播列表"来干什么？**这是把公会端接口换个前缀——**最危险的一种**	`GET /me/profile/detail`（只有他自己）
`POST /org/anchor/delete`	**硬删除**（合规和历史漏斗都要留）	`POST /org/anchor/remove`（软删）

B.2 四类前缀的边界（什么放哪、放错会怎样）

B.2.1 `/pub/*` —— 公开（不需要身份）

放什么：登录、验证码、入驻申请、健康检查、产品介绍页的数据。

接口	干什么
`POST /pub/auth/sendCode`	发验证码
`POST /pub/auth/login`	登录（返回 token + **可用身份列表**）
`POST /pub/auth/switchIdentity`	切换身份（一个人多个身份时）
`POST /pub/apply/submit`	提交入驻申请
`GET /pub/apply/status`	查申请进度（凭手机号 + 验证码）
`GET /pub/meta/health`	健康检查

放错的后果：把"要登录才能看"的东西放进 `/pub` → **不用登录就能拿到。**

判断标准：**这个接口，一个完全没有账号的人能不能调？** 不能 → 不许放 `/pub`。

B.2.2 `/org/*` —— 公会门（本公司的数据）

放什么：**只要涉及"这家公司的经营数据"的，全在这里。**

按业务分 9 组（见 B.4 清单）：组织与权限 · 主播与流水线 · 招聘面试 · 培训 · 形象与视觉 · 内容 · 排班与跟播 · 复盘与规划 · AI 对话与数据。

放错的后果：公会业务接口放进 `/me` → **主播就能看到公司的数据**（第二道防线靠字段白名单兜，但那已经是"补救"了）。

B.2.3 `/me/*` —— 主播门（★ 只有他自己）

放什么：主播自己的：资料、方案、复盘、开播前清单、对话、封面额度。

★ 这个前缀只有一条规矩，但它是全文档最重要的一条：

> ### ★★ 主播端接口一律 `/me/*`，**绝不复用 `/org/*` 不许做"同一个接口、用参数区分主子"这种事。 不许做"`/org/anchor/detail` 给主播也调一下、只是少返回几个字段"这种事。**

为什么这条不能商量（三个理由，一个比一个硬）：

#	理由
1	**复用 = "默认能看到"。** 一个被两边共用的接口，它的字段是"按最宽的那个角色设计的"——主播能看到什么，取决于**每次返回时有没有记得裁。**
2	**安全设计的默认值必须是"看不见"。** 分开写，主播那个接口的字段是**从零开始加的**（白名单）；共用的话，字段是**从一个已经很宽的结构里减的**（黑名单）。**减永**

	远比加容易漏。
3	**它让 CI 测试写得出来。** 有一条测试是"用主播身份调所有 `/org/*`，断言全部被拒"——**前提是两边真的分开。**共用了，这条测试就测不出东西（因为它本来就"合法"）。

影响什么商业结果：**这三条合起来就是 ToC 能不能成立。**

主播要是发现"公司那套系统能看到我的评级"——**他立刻明白这是监视工具，不是帮他赚钱的工具**（产品方案 7.1 三条硬纪律）。

B.2.4 `/platform/*` —— 平台方门（我们这一侧）

放什么：入驻审核、激活漏斗、公会健康度、成本与计量、合规台账、知识库与版本管理。

★ 这里有一条和 `/me` 对称的硬规矩：

> **平台方的"跨租户"能力，必须是"另一条路"，不是"同一个接口的开关"。** `/platform/*` 的查询**走单独的实现，不是在通用查询里开一个"不带 tenant_id"的开关。 而且输出前必须脱敏**——只出汇总，不出明细内容（产品方案 3.6：看体温，不看病历）。

放错的后果：一旦在通用查询里允许"不带 `tenant_id`"，

任何一个开发者写漏条件，就是跨租户泄露（技术方案 11 节 A.6 第 3 条）。

B.3 三条硬规矩（写进代码评审清单）

#	规矩	违反的后果
1	**★ 主播端一律 `/me/*`，绝不复用 `/org/*`**	四道防线的第一道被拆掉，ToC 立场崩
2	**`tenant_id` 只从登录态取，永远不从入参取**	前端传什么就能查什么 → 一次改包就跨租户
3	**每个接口必须声明"权限码"与"数据范围"**	权限判断散落各处（技术方案 3.1 就是防这个）

第 3 条具体怎么落（给 IT 的写法建议）：

每个接口的注册处（路由表里）**必须带三样元信息**：

```
接口：GET /org/anchor/detail
权限码：anchor.view          ← 对应 role_permission
数据范围：scope                ← 用登录态里的 rank 决定收窄到 本人/本组/本部门/全公司
敏感字段：按 field_tag 自动裁    ← 不需要写，数据出口统一做
```

开发者在写业务代码时，不需要知道权限怎么算—— 他只需要在注册接口时，把这三样标出来（技术方案 3.1：权限只在一处判断）。

B.4 接口清单（按业务分组）

读法：只写"有哪些接口、干什么用"。 **不写每个参数**——那是 IT 自己的产出。

B.4.1 公开（6 个）

接口	干什么
`POST /pub/auth/sendCode`	发验证码
`POST /pub/auth/login`	登录
`POST /pub/auth/switchIdentity`	切身份（一人多身份）
`POST /pub/apply/submit`	提交入驻申请（★ 5 个输入框，见产品方案 4.2）
`GET /pub/apply/status`	查申请进度
`GET /pub/meta/health`	健康检查

B.4.2 公会 • 组织与权限（6 个）

接口	干什么
`GET /org/org/tree`	部门与组结构（画组织树）
`GET /org/staff/list`	员工列表（岗位/职级）
`GET /org/staff/detail`	员工详情
`POST /org/staff/update`	改岗位/职级/部门（**★ 客户自己配，不发版**）
`GET /org/role/list`	岗位列表（**系统预置 11 个 + 客户自定义**）
`POST /org/fieldPolicy/update`	**★ 改字段可见性**（"保底金额给谁看"——客户自己配）

★ 最后两个要单独说：它们把"岗位"和"字段标签"变成了**客户能自己改的数据。 这就是"加一家客户不用发版"的落点**——否则每来一家公会都要改一次代码。

B.4.3 公会 • 主播与七段流水线（8 个）

接口	干什么
`GET /org/anchor/list`	主播列表（**按范围自动收窄**；筛负责人/组/阶段/等级）
`GET /org/anchor/detail`	**★ 主播档案"一页看完"**（七段 + 四场记录 + 复盘 + 前面几段留下的）
`POST /org/anchor/create`	新建主播
`POST /org/anchor/update`	修改
`POST /org/anchor/submit`	**推进到下一段**（走闸门）
`POST /org/anchor/approve`	闸门审核通过
`POST /org/anchor/reject`	闸门驳回（带理由）
`POST /org/anchor/transfer`	转组 / 换负责运营

> **★ `detail` 这个接口是"一页看完"的技术落点**：它一次返回**六块数据**（基本/七段/最近四场/AI 复盘/前面几段/边界说明）。**为什么要有这么一个"聚合接口"，而不是让前端调六个接口**：① 手机上一屏要出结果，**六次往返太慢**；② **权限只在一次出口判**（六次就是六次判断，漏一次就出事）。

B.4.4 公会 · 招聘面试 + 培训（5 个）

接口	干什么
`GET /org/interview/list`	面试记录列表
`POST /org/interview/create`	新增面试记录（**★ AI 成稿：说两句就生成，不用填表**）
`GET /org/training/list`	培训记录与进度
`POST /org/training/submit`	提交培训成绩与答疑记录
`GET /org/handover/list`	交接包（经纪人 → 培训 → 运营，**信息不掉**）

B.4.5 公会 · 形象与视觉（4 个）

接口	干什么
`GET /org/styling/list`	妆造预约
`POST /org/styling/create`	约妆造
`GET /org/visual/detail`	直播间参数（**给到人，不写进设备**）
`POST /org/visual/update`	存一套参数（美颜/灯光/机位）

★ **这一组要写清一条边界**：**参数是"发给人的"，不是"写进设备的"。 没有"下发到设备"这个接口，也不会有**——美颜是平台 App 内部设置，第三方改不了（技术方案 6.3-E）。

B.4.6 公会 · 排班与跟播（6 个）

接口	干什么
`GET /org/schedule/list`	排期（直播间席位）
`POST /org/schedule/submit`	排班提交（**撞车当场拦**）
`GET /org/followup/list`	跟播记录列表
`POST /org/followup/create`	★ **跟播记录新建（5 个字段）**
`POST /org/followup/submit`	提交（**第 4 步**）
`POST /org/recap/generate`	★ **生成复盘**（读已有的妙记录音 → 结构化复盘 + 下场建议）

★ **`followup/create` + `submit` 是"数据真假的开关"**（产品方案 E-06）： **5 个字段、允许先空着、提交时和后台导出的数对一遍。 "允许空"不是宽容**——硬要求必填，人就会乱填（产品方案 6.1）。

B.4.7 公会 · AI 对话与数据（6 个）

接口	干什么
`POST /org/chat/send`	按角色的对话（**带提问者的权限上下文**）
`GET /org/chat/list`	历史对话
`POST /org/data/import`	导入（公会后台导出的表）
`GET /org/data/importStatus`	导入进度与失败行
`GET /org/kpi/snapshot`	一句话查数据（**给看板与问答共用**）
`POST /org/report/generate`	日报/周报生成

★ **`chat/send` 这一条要记牢**：**AI 的权限 = 提问者的权限。 不能给 AI"全公司权限"——那是绕开所有权限的后门**（技术方案 3.3 第 2 条）。

B.4.8 主播门（9 个）★ 全部在 `/me/*`

接口	干什么
`GET /me/profile/detail`	我的资料（**0 个必填字段**）
`POST /me/profile/update`	改资料

`GET /me/plan/current`	当前这场方案
`GET /me/plan/stage`	这 7 天的方案
`GET /me/plan/longterm`	长期成长路径
`GET /me/recap/latest`	最近一场复盘
`GET /me/checklist/today`	开播前清单（话术/红线/节奏/参数）
`POST /me/chat/send`	问一句
`GET /me/quota/image`	封面出图额度

★ 注意这一组里"没有"的东西：

没有	为什么
✕没有 `me/anchor/list`	主播不需要"主播列表"——**有了就说明复用错了前缀**
✕没有 `me/settlement/*`	结算、保底、分成，**主播端一律不放**
✕没有 `me/rating/*`	评级、去留、淘汰环节，**一律不放**
✕没有 `me/team/*`	别人的数据，**一律不放**

B.4.9 平台方门（14 个）

接口	干什么
`GET /platform/apply/list`	入驻申请池
`POST /platform/apply/approve`	审核通过（**一键开号 + 装行业默认模板**）
`POST /platform/apply/reject`	驳回（**带理由码，不手写**）
`GET /platform/tenant/list`	平台上的公会列表
`GET /platform/tenant/health`	公会健康度（登录人数/活跃岗位/用量/沉默天数）
`GET /platform/funnel/activation`	★ **激活漏斗**（申请→审核→开户→激活→深度使用）
`GET /platform/cost/overview`	成本总览（总花费/人均/按能力分类）
`GET /platform/cost/byTenant`	★ **按公会分摊**（**不对客户公开**）
`GET /platform/cost/byFeature`	按功能聚合（"钱花在哪个功能上"）
`GET /platform/cost/alerts`	异常消耗告警
`GET /platform/kb/list` / `POST /platform/kb/update`	行业知识库上下架
`GET /platform/version/list` / `POST /platform/version/update`	标准版本与功能开关
`GET /platform/compliance/ledger`	★ **合规台账**（DPA 签订状态）

`GET /platform/feedback/list`	反馈池（按多少家提排序）

★ **`cost/byTenant` 与 `cost/byFeature` 是"算力中转站"的第一块表**（技术方案 7）。**`byTenant` 回答"这家客户免费期烧了我多少钱"**（何时开始收费的依据）；**`byFeature` 回答"钱主要花在哪、该优化哪个"——没有它，优化是盲目的。**

B.5 ★ 给 IT 的自检：哪些东西绝对不能出现在 `/me/*` 里

这一节是整份附录最重要的部分。 它不是"注意一下"，是"写错了就是事故"。

B.5.1 绝对不能出现在 `/me/*` 的字段（主播身份一律不返回）

评级 / 面试评级 / 培训评级 / 去留状态 / 流失原因 / 淘汰环节 / 所在段位（stage）
保底金额 / 分成比例 / 公司流水 / 他人流水 / 他人联系方式
工资 / 绩效 / 组内排名 / 复盘录音原始文件 / 面试评价
其他主播的任何信息（含昵称）
公司层面的聚合数据（多少人、多少流水）

注意"所在段位（stage）"这一条：七段流水线本身是公司内部的进度。 **主播知道"我在第几段"没有坏处，但"淘汰环节"绝对不能给**—— 所以主播端**要么不给 stage，要么给一个脱敏的说法**（如"成长期/稳定期"）。 **这条要 IT 在实现时明确选一种，不许"顺手带出来"。**

B.5.2 绝对不能出现在 `/me/*` 的接口（这些动作主播端根本不该有）

✕不许有的	为什么
`me/anchor/list`	主播不该有"主播列表"——**这是公会端的形状**
`me/openAnchor`（开户/建主播）	建主播是公司动作
`me/schedule/submit`（排班）	排班是运营的事
`me/settlement/*`（结算）	钱的事，公司端才有
`me/audit/*`（审核）	闸门审核是公司动作
`me/checklist/submit`（代提交）	主播端**不产生"要交的活"**——他这里没有 KPI 表单

★ **最后一条值得单独说**：**主播端的产品定位是"帮他播得更好"，不是"给他派活"。 一旦主播端出现了"待办/待提交"这种东西，这个定位就歪了**—— 他会觉得"又多了一个要交的系统"，**而主播端最不能有的就是这种感觉**（产品方案 7.1）。

B.5.3 三条自动检查（写进 CI，每次提交跑）

#	检查	断言
1	用主播身份调**每一个** `/org/*` 接口	**全部被拒**（401/403），且日志里记下越权尝试
2	调 `/me/*` 接口，扫返回的 JSON	**不出现 B.5.1 清单里的任何字段名**
3	调任意接口，**故意在入参里带上别家的 `tenant_id`**	**被忽略**（以登录态为准），**不返回任何别家数据**

> **这三条就是"四道防线里第四道"的落地**（技术方案 2.5）： **前三道是"设计上不可能漏"，这三条是"证明它真的没漏"。 没有它们，前三道就只是"看起来安全"。**

B.6 接口数量与分批（给排期的）

组	数量	什么时候做
公开（`/pub`）	6	**第 1 批**（登录与入驻是入口）
公会 · 组织与权限（`/org`）	6	**第 1 批**（地基：权限与字段策略）
公会 · 主播与流水线	8	**第 2 批**（业务骨架）
公会 · 排班与跟播	6	**第 2 批**
公会 · AI 对话与数据	6	**第 2 批**
主播门（`/me`）	9	**第 3 批**（一期末；**但前缀和账号体系第 1 批就要留好**）
平台方门（`/platform`）	14	**第 1-2 批**（没有它，公会进不来、也管不住）
公会 · 招聘/培训/形象/内容	9	**第 3 批**（按一期清单）
合计	**约 64 个**	——

> ★ **口径说明**：**技术方案正文早期写过"约 150 个接口"**——那是按"每个数据实体 × 每种操作"估的规模感。 **按一期的功能清单收敛下来，实际约 64 个**（列表/详情/增改/提交各组大致对齐）。 **两个数说的是两件事**：150 是"做全了的规模"，64 是"一期要做的"。 （第 8 轮收口时统一过口径：**技术方案里凡是承诺性的数字，都以一期清单为准。**）

B.7 附录小结：这份清单定了之后，哪些事变成了"不可能"

#	变成了不可能的事	靠什么
1	**主播端拿到公司数据**	一律 `/me/*`，**不复用** `/org/*`（B.2.3）
2	**把主播端接口偷偷加到公会端路径下**	CI 第 1 条：用主播身份调所有 `/org/*`，断言全拒
3	**靠改入参看别家公司**	`tenant_id` 只从登录态取（B.3 第 2 条）
4	**主播端多返回一个字段**	字段白名单默认全禁 + CI 第 2 条扫 JSON
5	**平台方偷看客户明细**	`/platform/*` 走单独实现 + 输出前脱敏（B.2.4）
6	**AI 变成一个"全公司权限"的后门**	`chat/send` 必须带提问者权限上下文（B.4.7）
7	**加一家客户要改代码**	岗位与字段策略是数据（B.4.2 最后两条接口）
8	**主播端变成"又一个要交的系统"**	`/me/*` 里**不许有 submit/审核/结算类接口**（B.5.2）

> **这八条就是"新来的后端照着搭骨架"要守住的东西。 路径可以调、接口可以拆、参数可以改——这八条不能丢。**

本文档与 `1_产品方案.md`、`2_开发原型.html` 的一致性已逐条核对（第十一节）。

方法：调研摸排 → 第一性原理 → 对抗性审查 → 结论。凡查不到的，文中标"待确认"，没有编数字。